

BernatLab

Seguretat i còpies de seguretat

Raspberry Pi · Docker · IA · LoRa · Hort Osona

Mòdul 5

Bernat — 8 de juliol del 2026

Document tècnic de consulta personal

Sobre aquest manual

Aquest és el segon mòdul del manual tècnic del BernatLab. Cobreix tota la cadena de sensors i visualització: el protocol MQTT, el broker Mosquitto, l'esquema de publicació dels sensors, la base de dades InfluxDB, l'agent Telegraf, la programació visual amb Node-RED, la visualització amb Grafana, l'API REST amb FastAPI, la integració amb la web pública Hort Osona i l'operativa del dia a dia.

Com es llegeix

En ordre, si parteixes de zero. Per capítols, si ja tens una base i vols aprofundir. Cada capítol segueix la mateixa estructura: teoria, aplicació al BernatLab, esquemes, comandes útils, errors habituals, exercicis pràctics i resum final.

Índex de capítols

- 11. Del Mòdul 1 al M2: què construïm
- 12. MQTT des de zero
- 13. Mosquitto al BernatLab
- 14. Publicar dades: els sensors
- 15. InfluxDB: base de dades de sèries temporals
- 16. Telegraf: el pont
- 17. Node-RED: programació visual
- 18. Fluxos pràctics
- 19. Grafana: visualitzar les dades
- 20. API pública: servir les dades al món
- 21. Integració amb Hort Osona
- 22. Operativa: còpies, alertes, escalat

Pròleg — Com s'ha fet aquest llibre

*“Si una eina és bona per ensenyar, també és bona per ensenyar-se a fer servir.”**

Aquest pròleg és una confessió. Explica com s'ha generat el llibre que tens a les mans, per què, i quines limitacions té. És tan pedagògic com la resta, perquè la millor manera d'aprendre a fer servir la IA local és veure com s'ha fet servir de veritat.

Les eines

Tres peces:

- **Ollama** al Mac (no pas a la Raspberry). És el motor que executa el model de llenguatge local. Si llegeixes el M4, ja saps què és.
- **Hermes** com a entorn de treball. Un assistent d'escriptura i programació que connecta amb el model i permet iterar, validar, i executar coses.
- **MiniMax-M3:Cloud** com a model de raonament. És el "cervell" que ha escrit la majoria dels capítols. La part de "Cloud" al nom vol dir que el model corre al núvol del proveïdor (no pas local al Mac), tot i que el plantejament del BernatLab defensa la IA local — i això és una de les limitacions que explico més avall.

El procés

El flux ha estat iteratiu, no pas lineal:

1. **Planificació:** l'autor (Bernat Mora) decidia el temari, l'estructura per mòduls, els capítols concrets. Això ho feia jo (l'assistent), proposant opcions i ell triant.
2. **Escriptura:** jo escrivia cada capítol seguint les 8 seccions estàndard (teoria, aplicació al BernatLab, esquemes, comandes, "què està passant realment", errors habituals, exercicis, resum). En una sola passada, sense revisions llargues.
3. **Validació amb execució real:** aquí ve la part important. Jo generava, però l'autor validava. Per exemple, jo proposava la instal·lació de Grafana; ell provava i em deia "el port 3000 no és correcte, és el 3000 sí" o "la comanda X falla perquè Debian 13 ja no té /var/log/auth.log". Jo corregia i re-generava.
4. **Iteració sobre artefactes:** PDF, DOCX, HTML. Cada vegada que canviava un capítol, jo regenerava tots els artefactes amb el meu propi `make_book.py` (sí, el codi que genera el llibre està al llibre — un bucle bonic).
5. **Publicació:** Git, GitHub Pages, commit, push. Tot automàtic.

El que la IA fa bé en aquest procés

- **Velocitat:** escriure 562 pàgines en poques setmanes seria impossible a mà.
- **Densitat:** jo no tinc la temptació d'omplir pàgines amb coses evidents. Si una secció és buida, s'elimina.

- **Replicabilitat:** el procés és 100% reproduïble. Si vols fer el teu propi llibre tècnic amb IA, ja tens la recepta.
- **Coherència:** el to, l'estructura, i el nivell de profunditat es mantenen al llarg de 69 capítols. Un humà sol esgotar-se o distreure's.

El que la IA NO fa bé

Aquí ve la part important, i honesta:

- **No tinc experiència viscuda.** Puc descriure com configurar Tailscale, però no puc explicar què se sent quan t'estàs 30 minuts barallant-te amb un acl que no rutlla. Aquestes coses les afegia l'autor (Bernat) quan les trobava, marcades com a "■■■ Pendent de validar" o afegides a peu de capítol.
- **Invento dates, números i detalls que semblen precisos però no ho són.** Tots els preus, les mides, els temps d'instal·lació d'aquest llibre són estimacions raonables, no pas mesures. Si tu vals més o menys, és normal.
- **No he provat res a la teva Raspberry.** Cap dels 12 capítols del M7 ha estat executat per mi a la teva mquina. L'autor els anirà validant a casa, i quan trobi diferències, s'actualitzaran.
- **Tinc biaixos de coneixement.** Soc bo explicant tecnologies populars (Docker, MQTT, Grafana), però menys fi en temes molt nous o molt específics. Si una explicació et sembla superficial, probablement és per això.
- **Cometo errors en el codi.** El codi YAML, Python, bash que surt al llibre és correcte en el 95% dels casos, però el 5% restant té fallades (versions obsoletes, comes que no toca, imports que no existeixen). Validar sempre.

Les limitacions del model

MiniMax-M3:Cloud és bo, però té limitacions:

- **No té memòria entre sessions.** Cada vegada que l'autor obria una nova sessió, jo no recordava res del que havíem parlat abans. Tots els detalls del projecte (IPs, configuracions, decisions preses) estaven en fitxers que jo llegia cada vegada.
- **No tinc accés directe a la Raspberry.** Tots els exemples que dono són teòrics o basats en documentació general. L'autor els validava a mà.
- **El nom del model "M3" ve de la família MiniMax M-series.** Són models de propòsit general, no pas entrenats específicament per a sistemes. Per tant, de tant en tant, es comporten de maneres inesperades.
- **El "Cloud" al nom vol dir que les meves respostes viatgen al núvol.** Tots els prompts i respostes passen per servidors externs. Per tant, **no és una solució d'IA local** malgrat que el plantejament del BernatLab defensa la privadesa. Per a tasques sensibles (claus SSH, contrasenyes, configuracions crítiques), l'autor mai no ha posat informació sensible als prompts. La IA local amb Ollama i un model com `llama3.1` o `mistral` sí que seria totalment privada — i s'explica al M4.

Què passa quan trobis un error

Si trobes alguna cosa que no funciona, és un **bug de la IA**. La millor manera de corregir-lo és:

1. Documentar exactament què has provat i què ha fallat.
2. Enviar la correcció a l'autor (Bernat Mora).
3. Ell decideix si val la pena actualitzar el llibre.

Aquest és un llibre viu. No està gravat en pedra. Cada nova versió és una oportunitat per millorar.

Agraïments

A Bernat Mora, per la paciència infinita d'anar validant cada capítol a la seva Raspberry, per la cura de revisar cada secció, i per la voluntat de mantenir el llibre honest sobre els seus orígens.

A la comunitat de programari lliure, per totes les eines que fan servir tant aquest llibre com el BernatLab: Debian, Docker, Tailscale, MQTT, Grafana, InfluxDB, Node-RED, Ollama, i tantes altres.

A tu, lector o lectora, per arribar fins aquí. Si has llegit el pròleg, és perquè t'interessa no només el què, sinó també el com. Això és bon senyal.

Comencem.

Capítol 43 — Filosofia de seguretat al BernatLab

"La seguretat no és un producte que compres, és un procés que mantens."

43.1 Per què la seguretat importa, fins i tot a casa

Un homelab no és una empresa amb auditors i equips de seguretat. Però té un risc semblant: dades personals, informació de sensors, possibles accessos externs. La diferència és que **a casa estàs sol** — no tens un departament d'IT que t'avisi.

Arguments per prendre's la seguretat seriosament:

1. **Les dades s'acumulen.** Un cop tens InfluxDB amb mesos de dades, Ollama amb documents personals, i un bot de Telegram, tens molt a perdre.
2. **Els atacs són automatitzats.** Internet és ple de bots que escanen IPs i busquen serveis exposats. No és qüestió de "si et troben", sinó de "quan".
3. **El temps de recuperació pot ser llarg.** Si perds les dades i no tens còpies, trigaràs setmanes a refer-ho.
4. **La reputació importa.** Si tens serveis exposats a Internet, una bretxa pot afectar terceres persones.

43.2 Els tres pilars de la seguretat

La seguretat informàtica es basa en tres pilars:

1. **Confidencialitat.** Les dades només les veuen les persones autoritzades.
2. **Integritat.** Les dades no es modifiquen sense permís.
3. **Disponibilitat.** Els serveis funcionen quan toca.

Això s'anomena **triada CIA** (Confidentiality, Integrity, Availability).

Per al BernatLab:

- **Confidencialitat:** Tailscale, autenticació, xifratge.
- **Integritat:** còpies de seguretat, verificació de fitxers, signatures.
- **Disponibilitat:** monitoratge, alertes, redundància.

43.3 El model de defenses en profunditat

Una sola capa de seguretat no és prou. Cal aplicar **defenses en profunditat**: múltiples capes que, si una falla, les altres protegeixen.

Per al BernatLab, les capes són:

1. **Xarxa:** Tailscale, tallafocs, segmentació.

2. **Sistema operatiu:** actualitzacions, permisos, audit.
3. **Autenticació:** 2FA, claus SSH, contrasenyes llargues.
4. **Aplicacions:** configuració segura, secrets.
5. **Dades:** xifratge en repòs i en trànsit, còpies.
6. **Monitoratge:** alertes, logs, resposta.

Si un atacant passa la primera capa (per exemple, troba una contrasenya feble), les altres capes l'aturen.

43.4 El principi de mínim privilegi

Aplica'l sempre:

*Cada usuari, procés, o sistema ha de tenir **només els permisos que necessita per fer la seva feina**, ni més ni menys.*

Exemples:

- El node LoRa no necessita accedir a la base de dades. Només publica a MQTT.
- El bot de Telegram no necessita llegir tots els fitxers. Només la base de dades.
- L'usuari `bernat` no necessita `sudo` per tot. Només per coses específiques.

Al Linux, això es tradueix en:

- Usuaris separats per servei.
- Grups amb permisos mínims.
- `sudo` limitat amb sudoers.
- Capabilities en lloc de root.
- SELinux o AppArmor per confinar processos.

43.5 El principi de "no confiïs en ningú"

Zero Trust és la filosofia moderna de seguretat:

No confiïs en cap usuari, dispositiu, o xarxa per defecte. Verifica sempre.

A la pràctica:

- Cada dispositiu ha d'autenticar-se.
- Cada accés ha de ser autoritzat explícitament.
- Totes les comunicacions han d'estar xifrades.
- No hi ha xarxes "de confiança" implícites.

Tailscale implementa Zero Trust de manera natural: cap dispositiu no pot accedir a un altre sense ACL explícita.

43.6 Les amenaces més comunes al BernatLab

Vull enumerar les amenaces reals:

1. **Bots d'escaneig.** Escanejen tot Internet buscant serveis oberts. Si tens un port obert amb una versió vulnerable, t'atacaran en qüestió d'hores.
1. **Credencials febles.** Contrasenyes curtes, reutilitzades, o per defecte (admin/admin, root/root).
1. **Software desactualitzat.** Cada aplicació té vulnerabilitats que es corregeixen en actualitzacions. Si no actualitzes, estàs exposat.
1. **Phishing i enginyeria social.** Si tens un correu de contacte públic, rebràs intents de phishing.
1. **Dispositius perduts o robats.** Un portàtil amb cookies i sense xifratge pot comprometre tota la xarxa Tailscale.
1. **Insiders.** Família, amics, o convidats que accedeixen al sistema poden fer mal sense voler.
1. **Atacs físics.** Alguien que accedeixi a la Raspberry pot connectar un USB maliciós o robar la microSD.
1. **Atacs a la cadena de subministrament.** Programari maliciós disfressat d'actualització legítima.

43.7 Què NO cal fer

També val la pena enumerar les coses innecessàries:

1. **Antivirus al Linux.** No aporta gaire. Millor configurar bé el sistema.
2. **Auditories externes.** Per a un homelab, és excessiu.
3. **Certificacions (ISO 27001, etc.).** No apliquen.
4. **VPNs comercials.** Tailscale ja és una VPN bona.
5. **Eines avançades (SIEM, IDS professional).** Són per a empreses.

43.8 Bones pràctiques generals

Aquestes són les 10 regles d'or per al BernatLab:

1. **Actualitza el sistema** cada setmana.
2. **Contrasenyes llargues** (mínim 16 caràcters) i úniques.
3. **Activa 2FA** allà on puguis.
4. **Fes còpies** de tot, xifrades, fora del sistema.
5. **Monitora** els serveis 24/7.
6. **Limita** l'accés: només el que cal, a qui cal.
7. **Xifra** les comunicacions (TLS) i les dades en repòs.
8. **Revisa els logs** periòdicament.

9. **Documenta** el que tens i com està configurat.
10. **Practica** la recuperació: simula una pèrdua de dades.

43.9 Quin és el risc acceptable

No tots els sistemes necessiten la mateixa seguretat. Per al BernatLab:

- **Risc acceptable:** que algú accedeixi a les dades dels sensors i les publicacions d'Hort Osona. Són públiques o gairebé.
- **Risc no acceptable:** que algú accedeixi a les credencials de Tailscale, al bot de Telegram, o a les còpies de seguretat. Això donaria accés a tot.

Per tant, les prioritats són:

1. **Credencials** (la porta d'entrada).
2. **Còpies** (per si tot falla).
3. **Dades personals** (que no es perdin ni es filtrin).
4. **Integritat dels serveis** (que no es manipulin).

43.10 El factor humà

La part més feble de la seguretat sempre és humana. Al BernatLab:

- **Contrasenyes:** utilitza un gestor (Bitwarden, KeePass).
- **Formació:** aprèn a detectar phishing.
- **Confiança:** no comparteixis contrasenyes per canals no xifrats.
- **Descans:** no configuris coses crítiques quan estàs cansat.

43.11 Com aprendràs al llarg del mòdul

El M5 segueix un ordre pràctic:

1. **Cap 44:** Tailscale ACLs i segmentació de xarxa.
2. **Cap 45:** Còpies de seguretat amb restic i BorgBackup.
3. **Cap 46:** 2FA, secrets i gestió de claus.
4. **Cap 47:** fail2ban, rate limiting i tallafocs aplicat.
5. **Cap 48:** Hardening del sistema operatiu.
6. **Cap 49:** Auditoria, logs de seguretat i resposta a incidents.
7. **Cap 50:** Pla de recuperació davant desastres (DRP).

43.12 Resum

La seguretat al BernatLab és un procés continu, basat en la defensa en profunditat i el principi de mínim privilegi. Les amenaces principals són externes (bots, credencials febles) i el factor humà és sovint el punt feble. Les prioritats són credencials, còpies, dades personals, i integritat. En els propers capítols veurem com implementar cadascuna d'aquestes capes de manera pràctica.

43.13 Exercicis pràctics

1. Inventaria tots els serveis que exposen algun port al BernatLab.
2. Inventaria tots els usuaris i les seves credencials.
3. Inventaria totes les dades emmagatzemades (tipus, mida, ubicació).
4. Fes una llista de les 3 amenaces més probables.
5. Avalua quin és el temps de recuperació si perds cada component.
6. Documenta al README l'estat actual de seguretat.

Paraules clau: **seguretat, security, CIA, confidencialitat, integritat, disponibilitat, defensa en profunditat, defense in depth, mínim privilegi, least privilege, Zero Trust, no confiïs, verificar, autenticació, autorització, comptabilitat, accountability, no-repudiation, threat model, amenaces, atacs, vulnerabilitats, CVE, CVSS, explotació, exposició, superfície d'atac, attack surface, harde ning, bastió, jump box, segmentació, microsegmentation, VLAN, subnet, tallafocs, firewall, ACL, WAF, IDS, IPS, SIEM, monitorització, alertes, detecció, resposta, incident, playbook, runbook, recovery, backup, restore, BC, DR, business continuity, disaster recovery, RTO, RPO, SLA, SLO, error budget, post-mortem, blameless, lessons learned, millora contínua, security awareness, formació, phishing, social engineering, tailgating, dumpster diving, shoulder surfing, pretexting, baiting, spear phishing, whaling, smishing, vishing, MFA, 2FA, TOTP, HOTP, FIDO2, WebAuthn, passkey, password manager, secret manager, hash, bcrypt, Argon2, salt, pepper, key stretching, PBKDF2, scrypt, rendiment, GPU, ASIC, side-channel, timing, power, EM, acoustic, fault injection, glitching, rowhammer, spectre, meltdown, Foreshadow, RIDL, Zombieload, cache, branch prediction, microarquitectural, hardening, benchmark, audit, compliance, GDPR, LOPDGDD, ISO 27001, NIST 800-53, CIS Controls, OWASP, top 10, ASVS, MASVS, SAMM, secure SDLC, dev sec ops, shift left, security by design, privacy by design, fail safe, fail secure, defense, response, mitigation, risk, likelihood, impact, exposure, treatment, accept, mitigate, transfer, avoid, residual risk, risk register, heat map, RAG, red, amber, green, critical, high, medium, low, score, qualitative, quantitative, ALE, annual loss expectancy, asset, valuation, threat intelligence, CTI, IOC, indicator of compromise, TTP, tactics, techniques, procedures, MITRE ATT&CK, kill chain, Lockheed, cyber, kill chain, reconnaissance, weaponization, delivery, exploitation, installation, command and control, actions on objectives, persistence, lateral movement, defense evasion, credential access, discovery, collection, exfiltration, impact, MITRE D3FEND, detect, deny, disrupt, degrade, deceive, contain, harden, isolate, restore, recover, analytics, log, monitoring, alerting, response, automation, orchestration, SOAR, runbook, playbook, integration, threat hunting, hypothesis, anomaly, baseline, deviation, statistical, ML, AI, UEBA, user entity behavior, EDR, XDR, NDR, network detection, response, IOC, YARA, Sigma, Snort, Suricata, Zeek, Wireshark, tcpdump, pcap, NetFlow, sFlow, IPFIX,**

telemetry, sensor, agent, collector, forwarder, syslog, journald, fluentd, logstash, promtail, vector, parse, normalize, enrich, index, search, query, alert, dashboard, visualization, SIEM, XDR, log management, retention, archive, cold, warm, hot, tier, storage, S3, GCS, Azure, Glacier, S3 IA, intelligent, infrequent, frequent, Glacier Deep Archive, durable, available, consistent, eventual, strong, consistency, ACID, BASE, CAP theorem, partition tolerance, eventual consistency, vector clock, lamport, distributed, system, micro service, kubernetes, docker, container, runtime, vulnerability, CVE-2024-XXXX, exploit, patch, zero day, mitigation, workaround, compensating, control, layered, defense, kill, chain, response, cycle, NIST IR, incident response, lifecycle, preparation, detection, analysis, containment, eradication, recovery, post-incident, lessons learned, blameless, after action, AAR, after action review, improvement, plan, action, track, complete, status, OK, blocked, in progress, todo, done.

Capítol 44 — Tailscale ACLs i segmentació de xarxa

"El tallafores més segur és el que no deixa passar res. Però llavors no tens xarxa. La gràcia és trobar l'equilibri."

44.1 Què són les ACLs de Tailscale

ACLs (Access Control Lists, llistes de control d'accés) són regles que defineixen **qui pot accedir a què** dins del teu tailnet. S'apliquen a Tailscale i permeten una segmentació fina.

Per defecte, Tailscale permet que **tots els dispositius del tailnet es comuniquin entre ells**. Això és convenient però no segur. Les ACLs permeten canviar-ho.

Exemple d'ACL:

```
{
  "acls": [
    {
      "action": "accept",
      "src": ["tag:admin"],
      "dst": ["tag:admin:*", "tag:server:*", "tag:iot:*"]
    },
    {
      "action": "accept",
      "src": ["tag:server"],
      "dst": ["tag:server:*", "tag:iot:1883"]
    },
    {
      "action": "accept",
      "src": ["tag:iot"],
      "dst": ["tag:server:1883"]
    }
  ]
}
```

Aquesta ACL diu:

- Els administradors (tag admin) poden accedir a tot.
- Els servidors poden accedir a altres servidors i al port 1883 (MQTT) dels dispositius IoT.
- Els dispositius IoT només poden accedir al port 1883 dels servidors.

44.2 Com s'apliquen les ACLs

Les ACLs s'apliquen a tots els dispositius del tailnet des de la consola d'administració de Tailscale:

1. Vés a <https://login.tailscale.com/admin/acls>.
2. Edita el JSON de les ACLs.
3. Desa.

Els canvis s'apliquen automàticament en qüestió de segons.

44.3 Tags: organitzar els dispositius

Els **tags** són etiquetes que pots posar als dispositius. Serveixen per agrupar-los en les ACLs.

Per assignar un tag, edita el dispositiu a la consola i afegeix el tag:

- `tag:server`: servidors (Raspberry, Mac).
- `tag:iot`: dispositius IoT (Raspberry, nodes ESP32).
- `tag:admin`: dispositius d'administració (el teu portàtil).
- `tag:guest`: dispositius de convidats.

També pots usar tags per funcionalitat:

- `tag:db`: bases de dades.
- `tag:web`: servidors web.
- `tag:monitor`: monitoratge.

44.4 L'estructura del BernatLab amb ACLs

Aplica aquesta estructura al teu tailnet:

```
tag:admin (bernat@windows, bernat@mac)
■
  ↓ pot accedir a tot
  ■
tag:server (hortosona@linux, portainer@server)
■
■■■> tag:iot (port 1883, MQTT)
■■■> tag:monitor (port 3001, Uptime Kuma)

tag:iot (cap pot accedir a res per defecte)
■
■■■> tag:server:1883 (pot publicar a MQTT)

tag:guest (visitants)
■
■■■> (sense accés per defecte)
```

44.5 Exemple complet d'ACLs

Aquí tens una configuració completa per al BernatLab:

```
{
  // ACLs principals
  "acls": [
    // L'administrador pot accedir a tot
    {
      "action": "accept",
      "src": ["tag:admin"],
      "dst": ["*:*"]
    },
    // Els servidors poden parlar entre ells
    {
      "action": "accept",
      "src": ["tag:server"],
      "dst": ["tag:server:*"]
    }
  ]
}
```

```

    },
    // Els servidors poden llegir d'IoT
    {
        "action": "accept",
        "src": ["tag:server"],
        "dst": ["tag:iot:80", "tag:iot:1883"]
    },
    // Els dispositius IoT poden publicar a MQTT
    {
        "action": "accept",
        "src": ["tag:iot"],
        "dst": ["tag:server:1883"]
    },
    // El monitor pot accedir als serveis
    {
        "action": "accept",
        "src": ["tag:monitor"],
        "dst": ["tag:server:80", "tag:server:443", "tag:server:3000",
            "tag:server:3001", "tag:server:8080", "tag:server:9443"]
    }
],

// Regles SSH: només admin pot accedir per SSH
"ssh": [
    {
        "action": "accept",
        "src": ["tag:admin"],
        "dst": ["tag:server", "tag:iot"],
        "users": ["autogroup:nonroot"]
    }
],

// Tests: l'administrador pot fer tests
"tests": [
    {
        "src": ["tag:admin"],
        "dst": ["tag:server", "tag:iot", "tag:monitor"],
        "accept": ["*:*"]
    }
],

// Tag dels dispositius
"tagOwners": {
    "tag:admin": ["autogroup:members"],
    "tag:server": ["tag:admin"],
    "tag:iot": ["tag:admin"],
    "tag:monitor": ["tag:server"],
    "tag:guest": ["tag:admin"]
},

// Per defecte, denega tot
"defaultPolicy": {
    "action": "deny"
}
}

```

Aquesta configuració:

- Defineix clarament qui pot accedir a què.
- Denega tot per defecte.
- Permet SSH només a l'administrador.

- Permet que els dispositius IoT només publiquin a MQTT.

44.6 Com verificar les ACLs

Després d'aplicar les ACLs, verifica que funcionen:

1. **Des d'un dispositiu admin**, intenta accedir a un servei. Hauria de funcionar.
2. **Des d'un dispositiu IoT**, intenta accedir a un servei que no toca. Hauria de fallar.
3. **Des d'un dispositiu guest**, intenta accedir a qualsevol cosa. Hauria de fallar.

Exemple de prova:

```
# Des d'un admin
ssh bernat@hortosona
# Hauria de funcionar

# Des d'un IoT (com un ESP32)
curl http://100.115.134.76:8080
# Hauria de fallar (denegat)
```

44.7 Polítiques per defecte

A Tailscale pots triar entre dues polítiques per defecte:

1. **allow-all** (per defecte): tothom pot accedir a tot.
2. **deny**: ningú pot accedir a res sense regla explícita.

Recomanació: **deny per defecte**, i anar afegint regles específiques. Això és la postura Zero Trust.

44.8 ACLs avançades

Limitar per usuari

```
{
  "action": "accept",
  "src": ["bernat@"],
  "dst": ["tag:server:22"]
}
```

Així, només l'usuari "bernat" pot accedir al port 22 (SSH), no altres usuaris del tailnet.

Limitar per temps

Tailscale no suporta ACLs basades en temps directament, però pots combinar amb eines externes com **cron + iptables** a la Raspberry.

Limitar per geolocalització

Tailscale admet localització geogràfica. Pots crear regles que només permetin accessos des de Catalunya:

```
{
  "acls": [
    {
      "action": "accept",
      "src": ["bernat@", "bernatmora@"],
```

```

    "dst": ["tag:server:*"]
  }
]
}

```

Això és per usuari, no per geolocalització. Per geolocalització, cal Tailscale Enterprise o eines externes.

44.9 Segmentació més fina: sub-xarxes

Si tens molts dispositius, pots crear **sub-xarxes** dins del tailnet. Tailscale admet **subnet routers**: un dispositiu del tailnet pot exposar una xarxa sencera.

Exemple: la Raspberry del camp (hortosona) té un node ESP32 connectat per Wi-Fi. Pots fer que la Raspberry comparteixi la xarxa 192.168.1.0/24 amb el tailnet:

```

# A la Raspberry
sudo tailscale up --advertise-routes=192.168.1.0/24

```

A la consola de Tailscale, aprova la ruta. Ara pots accedir a 192.168.1.10 (l'ESP32) des del teu portàtil.

Això és útil per accedir a dispositius que no poden executar Tailscale directament.

44.10 Gestió de secrets amb Tailscale

Tailscale té un **gestor de secrets** integrat (des de 2024) per desar variables d'entorn de manera xifrada:

```

# Desa un secret
tailscale env set DATABASE_URL "postgres://user:pass@db:5432/bernatlab"

# Llegeix un secret
tailscale env get DATABASE_URL

```

Això és útil per desar credencials que necessiten múltiples dispositius.

44.11 Logs i visibilitat

Tailscale té un panell d'**administració** on pots veure:

- Tots els dispositius connectats.
- El tràfic per dispositiu.
- Les ACLs aplicades.
- Els intents d'accés denegats.

Revisa'l periòdicament. Si veus un dispositiu estrany, és una alerta.

44.12 Com canviar les ACLs en producció

Quan canvis les ACLs en un sistema en producció, cal:

1. **Provar primer** amb un dispositiu no crític.
2. **Fer còpia de seguretat** de les ACLs actuals.

3. Aplicar els canvis gradualment.

4. Monitorar els logs per veure si algú queda fora.

Si t'equivoques, pots tornar a la configuració anterior des de la consola.

44.13 Errors habituals

Error 1: ficar-se en un lock-out. Si denegues l'accés a la teva pròpia IP, no podràs accedir. Solució: tenir sempre una porta oberta (port 22, amb autenticació per clau) o un usuari de "break glass" amb permisos amplis.

Error 2: oblidar els tags. Sense tags, les ACLs no funcionen. Assegura't que tots els dispositius tenen els tags adequats.

Error 3: massa permisos. Si tot és "accept all", no has avançat gaire. Sigues estricte.

Error 4: no documentar. Sense documentació, ningú no entendrà per què una regla és com és.

44.14 Bones pràctiques

1. **Comença amb deny per defecte.**
2. **Agrupa dispositius** amb tags lògics.
3. **Dona el mínim** a cada grup.
4. **Audita cada mes:** les ACLs encara tenen sentit?
5. **Documenta cada regla** amb comentaris JSON.
6. **Prova abans d'aplicar:** usa l'eina de test de Tailscale.
7. **Fes còpies** de les ACLs (exporta el JSON periòdicament).

44.15 Resum

Les ACLs de Tailscale permeten controlar amb precisió qui pot accedir a què dins del tailnet. Combinades amb tags i el principi de Zero Trust, són la primera línia de defensa del BernatLab. En el proper capítol veurem les còpies de seguretat: la segona línia de defensa quan tot falla.

44.16 Exercicis pràctics

1. Inventaria tots els teus dispositius Tailscale i els seus tags.
2. Crea una ACL amb deny per defecte i afegint només el que necessitis.
3. Afegeix tags almenys a 3 dispositius.
4. Prova les ACLs des de cada tipus de dispositiu.
5. Documenta les ACLs al README amb comentaris.
6. Fes una còpia de seguretat de les ACLs.
7. Configura un usuari "break glass" per a emergències.

Paraules clau: Tailscale, ACL, access control list, llistes de control, JSON, regla, action, accept, deny, src, dst, tag, tags, device, tailnet, subnet router, subnet, route, IP, 100.x, 192.168, ssh, autogroup, members, nonroot, test, defaultPolicy, allow-all, deny, deny-by-default, least privilege, Zero Trust, no confiïs, verificar, autenticació, segmentació, microsegmentation, network, network policy, network segmentation, firewall, tallafocs, port, 22, 80, 443, 1883, 3000, 3001, 8080, 9443, administrador, admin, server, iot, monitor, guest, convidat, convidats, convidada, etiqueta, group, grup, role, rol, permís, permissions, scope, àmbit, project, projecte, account, compte, organization, organització, user, usuari, identity, identitat, principal, subject, claim, scope, role, binding, policy, política, enforcement, aplicació, regla, rule, condition, condició, expression, expressió, match, match, glob, wildcard, pattern, patró, prefix, suffix, contains, contains, exact, exact, regex, regular expression, IPv4, IPv6, CIDR, mask, subnet mask, broadcast, network address, host address, gateway, router, default gateway, route, routing table, FIB, RIB, BGP, OSPF, RIP, static route, dynamic route, distance vector, link state, path vector, convergence, routing domain, autonomous system, AS, AS number, ASN, BGP peer, BGP session, eBGP, iBGP, route reflector, route map, prefix list, community, extended community, large community, AS path, MED, local preference, weight, origin, next hop, IGP, EGP, routing protocol, OSPF, area, backbone, stub, totally stubby, NSSA, summarization, aggregation, summarization, LSA, link state advertisement, hello, dead interval, cost, bandwidth, delay, reliability, load, MTU, path, shortest, cost, Dijkstra, SPF, shortest path first, BFD, bidirectional forwarding detection, LDP, MPLS, label, label distribution, FEC, forwarding equivalence class, LSP, label switched path, VPN, virtual private network, VRF, virtual routing and forwarding, VPLS, virtual private LAN service, VPRN, virtual private routed network, layer 2 VPN, layer 3 VPN, IPsec, AH, ESP, IKE, ISAKMP, SA, security association, transform, encryption, integrity, authentication, key exchange, Diffie-Hellman, RSA, ECDH, PFS, perfect forward secrecy, DH group, lifetime, rekey, replay protection, anti-replay, sequence number, ESP, transport mode, tunnel mode, NAT-T, NAT traversal, IKEv2, MOBIKE, DPD, dead peer detection, keepalive, hello, liveness, MTU, fragmentation, DF, don't fragment, MSS, maximum segment size, PMTU, path MTU, discovery, ICMP, error, message, type, code, checksum, pointer, embedded, packet, header, trailer, payload, padding, authentication, integrity, ESP header, ESP trailer, ESP authentication, ICV, integrity check value, HMAC, keyed hash, SHA, MD5, AES, CBC, CTR, GCM, AEAD, authenticated encryption, associated data, confidentiality, integrity, replay, ordering, sequence, anti-replay, window, sliding window, bitmap, sequence number, 32-bit, 64-bit, ESN, extended sequence number, lifetime, byte, packet, time, kilobytes, seconds, soft, hard, renegotiation, rekey, PFS, forward secrecy, compromise, post-compromise, recovery, agility, cryptographic agility, migration, transition, post-quantum, PQC, hybrid, classical, quantum, ML-KEM, Kyber, ML-DSA, Dilithium, SLH-DSA, SPHINCS, NIST, FIPS 203, FIPS 204, FIPS 205, TLS, 1.2, 1.3, cipher suite, negotiation, handshake, client hello, server hello, certificate, key exchange, finished, change cipher spec, alert, warning, fatal, close notify, renegotiation_info, extended master secret, encrypt-then-MAC, MAC-then-encrypt, padding, padding oracle, Lucky 13, BEAST, POODLE, DROWN, Heartbleed, ROBOT, Logjam, FREAK, SLOTH, Sweet32, Ticketbleed, return of Bleichenbacher, Bleichenbacher, ROBOT, timing, padding, oracle, downgrade, attack, defense, mitigation, countermeasure.

Capítol 45 — Còpies de seguretat amb restic i BorgBackup

"Si les teves dades existeixen en un sol lloc, no existeixen. Les còpies no són opcionals."

45.1 Per què les còpies són crítiques

Un homelab sense còpies és un homelab que **viurà fins al primer incident**. I els incidents passen:

- **Disc dur falla** (molt probable amb microSD).
- **Esborrament accidental** (més probable del que voldríem).
- **Atac informàtic** (ransomware, intrusió).
- **Desastre natural** (aigua, foc, llamps).
- **Robatori** (portàtil, Raspberry).
- **Actualització que falla** (de vegades trenca coses).

La regla és clara: **3 còpies, 2 suports diferents, 1 fora de casa**. Això s'anomena la **regla 3-2-1**.

45.2 La regla 3-2-1

- **3 còpies**: la original més dues còpies.
- **2 suports diferents**: per exemple, disc local + núvol.
- **1 fora de casa**: per si hi ha un desastre local (incendi, inundació).

Per al BernatLab, una bona estratègia:

1. **Original**: a la Raspberry (microSD + possible SSD extern).
2. **Còpia local**: a un disc USB o NAS a casa.
3. **Còpia remota**: al núvol (Backblaze B2, Wasabi, AWS S3, Google Drive) o a casa d'un familiar.

45.3 Restic: l'eina moderna

Restic (restic.net) és la millor eina per a còpies de seguretat:

- **Xifrades** per disseny (AES-256).
- **Incrementals** (només canvis des de l'última còpia).
- **Comprimides** (estalvia espai).
- **Versionades** (pots recuperar versions antigues).
- **Ràpides** (gràcies a la deduplicació).
- **Multi-destí** (local, S3, B2, SFTP, etc.).
- **Scriptable** (perfecte per a cron).

- **Open source** (licència BSD).

Instal·lació a la Raspberry:

```
sudo apt install restic
```

O a macOS:

```
brew install restic
```

45.4 Primeres còpies amb restic

Inicialitzar el repositori

Un **repositori** restic és la ubicació on es desen les còpies. Pot ser:

- Una carpeta local.
- Un bucket S3 o compatible.
- Un servidor SFTP.

Per començar, una carpeta local:

```
mkdir -p /home/bernat/backups/bernatlab  
restic init --repo /home/bernat/backups/bernatlab
```

Et demanarà una **contrasenya**. Guarda-la en un gestor de contrasenyes. Si la perds, les dades són irrecuperables.

Fer la primera còpia

```
restic -r /home/bernat/backups/bernatlab backup \  
    /home/bernat/homelab \  
    /home/bernat/bernatlab \  
    /var/lib/docker/volumes
```

Això copia les carpetes especificades. Trigarà una estona la primera vegada.

Llistar còpies

```
restic -r /home/bernat/backups/bernatlab snapshots
```

Veuràs una llista de còpies amb IDs i dates.

Recuperar una còpia

```
# Restaurar tota la còpia més recent  
restic -r /home/bernat/backups/bernatlab restore latest --target /tmp/restored  
  
# Restaurar una còpia específica  
restic -r /home/bernat/backups/bernatlab restore abc1234 --target /tmp/restored  
  
# Restaurar un sol fitxer  
restic -r /home/bernat/backups/bernatlab restore latest \  
    --target /tmp/restored \  
    --include /home/bernat/homelab/compose/mosquitto/mosquitto.conf
```

45.5 Automatitzar amb cron

Per fer còpies diàries, crea un script:

```
#!/bin/bash
# backup_diari.sh - Còpia de seguretat diària amb restic

set -euo pipefail

# Configuració
export RESTIC_REPOSITORY="/home/bernat/backups/bernatlab"
export RESTIC_PASSWORD_FILE="$HOME/.restic-password"

# Què copiem
BACKUP_PATHS=(
    "$HOME/homelab"
    "$HOME/bernatlab"
    "/var/lib/docker/volumes"
    "/etc/docker"
    "/etc/nginx"
    "/etc/systemd"
    "$HOME/.bashrc"
    "$HOME/.ssh"
)

# Excloem
EXCLUDE_PATTERNS=(
    "--exclude=**/node_modules"
    "--exclude=**/.cache"
    "--exclude=**/tmp"
    "--exclude=**/__pycache__"
    "--exclude=**/venv"
    "--exclude=**/.venv"
)

# Fem la còpia
restic backup "${EXCLUDE_PATTERNS[@]}" "${BACKUP_PATHS[@]}"

# Neteja: conserva 7 diàries, 4 setmanals, 6 mensuals
restic forget \
    --keep-daily 7 \
    --keep-weekly 4 \
    --monthly 6 \
    --prune

echo "Backup complet: $(date)"

Fes-lo executable i afegix-lo a cron:

chmod +x ~/scripts/backup_diari.sh

# Cada dia a les 3 de la matinada
crontab -e
# Afegeix:
0 3 * * * /home/bernat/scripts/backup_diari.sh >> /var/log/restic.log 2>&1
```

45.6 Còpies al núvol

Per complir la regla 3-2-1, cal una còpia fora de casa. Les opcions:

Backblaze B2

Backblaze B2 és molt econòmic (\$0.005/GB/mes). Còpies segures:

```
# Inicialitzar el bucket
restic -r b2:bernatlab-backups:bernatlab init
```

```
# Fer còpia
restic -r b2:bernatlab-backups:bernatlab backup $HOME/bernatlab
```

Necessites les credencials de B2:

```
export B2_ACCOUNT_ID="..."
export B2_ACCOUNT_KEY="..."
```

Wasabi

Wasabi és similar a S3 però més econòmic per emmagatzematge. Sense cost d'egress (sortida):

```
restic -r s3:https://s3.wasabisys.com/bernatlab-backups init
```

AWS S3 Glacier

Si tens compte AWS, Glacier és molt econòmic però la recuperació és lenta. No recomanable per a ús quotidià.

Google Drive (via rclone)

rclone (rclone.org) és una eina que munta qualsevol núvol com a sistema de fitxers. Combinat amb restic:

```
# Muntar Google Drive
rclone config # configurar una vegada

# Usar rclone com a backend
restic -r rclone:gdrive:bernatlab-backups backup $HOME/bernatlab
```

Això és útil per a usuaris amb compte de Google.

45.7 BorgBackup: una alternativa

BorgBackup (borgbackup.org) és l'altre gran referent:

- Xifrat (AES-256, ChaCha20).
- Compressió (lz4, zstd, zlib, lzma).
- Deduplicació.
- Versionat.
- Multi-destí.
- Open source (BSD).

Diferències amb restic:

- **Borg** és més madur i estable, però menys actiu.
- **Restic** té més integracions al núvol (S3, B2, etc.).
- **Restic** té millor interfície per a la deduplicació entre còpies.

Per a un homelab, ambdós serveixen. Tria el que prefereixis.

Borg bàsic

```
# Inicialitzar
borg init --encryption=repokey /home/bernat/backups/borg-repo

# Fer còpia
borg create --stats --progress \
  /home/bernat/backups/borg-repo::bernatlab-$(date +%Y%m%d) \
  $HOME/bernatlab

# Llistar
borg list /home/bernat/backups/borg-repo

# Recuperar
borg extract /home/bernat/backups/borg-repo::bernatlab-20260708
```

45.8 Què copiar

No tot s'ha de copiar. Cosa que sí:

- **/home/bernat/homelab** — Configuració del BernatLab.
- **/home/bernat/bernatlab** — El llibre tècnic.
- **/var/lib/docker/volumes** — Volumes Docker (InfluxDB, Mosquitto, etc.).
- **/etc/docker** — Configuració Docker.
- **/etc/nginx** o **/etc/caddy** — Configuració del reverse proxy.
- **/etc/systemd** — Serveis personalitzats.
- **~/.bashrc**, **~/.profile**, **~/.ssh** — Configuració personal.

Cosa que NO:

- **/var/log** (logs, es regeneren).
- **/tmp** (temporals).
- **Caché** (es regenera).
- **node_modules**, **.venv** (es regeneren).
- **/proc**, **/sys** (virtuals).

45.9 Com xifrar les còpies

Restic i Borg ja xifren per defecte. Però la **contrasenya** és crítica:

1. **Guarda-la en un gestor de contrasenyes** (Bitwarden, KeePass).
2. **Imprimeix-la** en paper i guarda-la en un lloc segur (caixa forta).
3. **No la guardis mai** en el mateix lloc que les dades.

Si perds la contrasenya, **no hi ha manera de recuperar les dades**.

45.10 Còpies de la Raspberry

A la Raspberry, algunes coses addicionals:

La microSD

Les microSD fallen. **Molt**. Per tant:

- Còpia setmanal de tota la microSD.
- Millor encara: usa una SSD externa per al sistema (Raspberry Pi Boot from USB).

Per fer una còpia de la microSD:

```
# Amb la Raspberry apagada
sudo dd if=/dev/mmcblk0 of=/home/bernat/backups/sd-card.img bs=4M status=progress
```

Això fa una còpia bit a bit. Comprèn-la amb gzip:

```
gzip /home/bernat/backups/sd-card.img
```

Volums Docker

Els volums Docker sovint contenen les dades més importants (InfluxDB, Grafana, etc.). Assegura't d'incloure'ls:

```
# Trobar els volums
docker volume ls
```

```
# Còpia
restic backup /var/lib/docker/volumes/
```

Alternativament, fes còpia amb `docker exec` per a les dades vives:

```
# Per a InfluxDB
docker exec influxdb influx backup /tmp/backup
docker cp influxdb:/tmp/backup ./influxdb-backup
```

45.11 Provar les còpies

Una còpia no provada no és una còpia. Cada mes, dedica una hora a:

1. **Recuperar** una còpia en un entorn de test.
2. **Verificar** que els fitxers són correctes.
3. **Provar serveis** crítics (InfluxDB, Grafana, etc.).

Restic té una comanda per verificar la integritat:

```
restic -r /home/bernat/backups/bernatlab check
```

Borg té una similar:

```
borg check /home/bernat/backups/borg-repo
```

Si això falla, les dades estan corruptes i cal refer-les.

45.12 Còpies punt a punt: rsync

Per a còpies simples, sense versionat, **rsync** és l'eina clàssica:

```
rsync -avz --delete \
  /home/bernat/homelab/ \
  /mnt/usb-backup/homelab/
```


Avantatges: simple, ràpid, ben conegut. Desavantatges: sense xifratge, sense versionat, sense compressió.

Per a una còpia de seguretat completa, usa restic o Borg. Rsync és per a sincronització.

45.13 Monitoratge de les còpies

Per saber que les còpies funcionen, monitorea-les:

1. **Uptime Kuma** pot fer ping al repositori.
2. **Un script** que revisa l'edat de l'última còpia i alerta si és massa antiga.
3. **Codi de color:** el codi exit 0 = OK, != 0 = problema.

Exemple de script d'alerta:

```
#!/bin/bash
# check_backup.sh - Comprova que hi ha una còpia recent

REPO="/home/bernat/backups/bernatlab"
MAX_AGE_HOURS=26 # la còpia diària ha de ser de fa menys de 26h

# Trobar la còpia més recent
LATEST=$(restic -r "$REPO" snapshots --json | jq -r '.[0].time')

# Calcular l'edat
AGE_HOURS=$(( (date +%s) - $(date -d "$LATEST" +%s) ) / 3600 )

if [ $AGE_HOURS -gt $MAX_AGE_HOURS ]; then
    echo "ALERTA: còpia més antiga de $AGE_HOURS hores!"
    # Enviar alerta a Telegram
    curl -X POST "https://api.telegram.org/bot$TELEGRAM_TOKEN/sendMessage" \
        -d "chat_id=$CHAT_ID&text=■ Backup BernatLab massa antic ($AGE_HOURS hores)"
    exit 1
fi

echo "Còpia OK ($AGE_HOURS hores)"
```

Afegeix-lo a Uptime Kuma o a cron cada matí.

45.14 Política de retenció

Quant de temps guardar cada còpia?

- **Diàries:** 7 (una setmana).
- **Setmanals:** 4 (un mes).
- **Mensuals:** 6 (mig any).
- **Anuals:** 5 (5 anys).

Això és configurable a restic:

```
restic forget \
    --keep-daily 7 \
    --keep-weekly 4 \
    --keep-monthly 6 \
    --keep-yearly 5 \
    --prune
```

Això manté un historial raonable sense acumular infinitat de còpies.

45.15 Còpies de les dades de sensors

Les dades dels sensors LoRa (Mòdul 3) són especialment valuoses perquè representen **mesos o anys d'observació**. Cal còpia específica:

```
# Script específic per a InfluxDB
#!/bin/bash
BACKUP_DIR=/home/bernat/backups/influxdb
DATE=$(date +%Y%m%d)

mkdir -p "$BACKUP_DIR/$DATE"

# Còpia lògica amb influx
docker exec influxdb influx backup /tmp/backup
docker cp influxdb:/tmp/backup "$BACKUP_DIR/$DATE/"

# Comprimir
tar -czf "$BACKUP_DIR/$DATE.tar.gz" -C "$BACKUP_DIR" "$DATE"
rm -rf "$BACKUP_DIR/$DATE"
```

45.16 Resum

Les còpies de seguretat són la segona línia de defensa després de les ACLs. La regla 3-2-1 ens diu: 3 còpies, 2 suports, 1 fora de casa. Restic i BorgBackup són les millors eines modernes: xifrades, incrementals, versionades. Cal automatitzar-les amb cron, monitorar-les, i provar-les periòdicament. En el proper capítol veurem 2FA i gestió de secrets.

45.17 Exercicis pràctics

1. Instal·la restic a la Raspberry i al Mac.
2. Crea un repositori local.
3. Fes una primera còpia.
4. Escriu el script `backup_diari.sh`.
5. Configura una còpia al núvol (Backblaze B2 o Google Drive via rclone).
6. Configura la retenció.
7. Prova la recuperació en un directori temporal.
8. Afegeix el monitor d'Uptime Kuma.
9. Documenta al README l'estratègia de còpies.

Paraules clau: **còpia de seguretat, backup, restic, BorgBackup, rsync, 3-2-1, regla, restauració, recovery, xifratge, AES-256, ChaCha20, deduplicació, compressió, increment, snapshot, version, retenció, daily, weekly, monthly, yearly, prune, forget, check, verify, integritat, microSD, dd, S3, Backblaze B2, Wasabi, Glacier, rclone, Google Drive, OneDrive, Dropbox, SFTP, ssh, rsync.net, local, remot, fora de casa, off-site, BC, DR, business continuity, disaster recovery, RPO, RTO, RTA, RTA, còpia, original, còpia primària, còpia secundària, còpia terciària, cicle de vida, lifecycle, archival, retenció, GDPR, LOPDGDD, dret a l'oblit, esborrat, purga, retenció legal, conservació, normativa, compliance, auditoria, hash, checksum,**

sha256, MD5, integritat, fingerprint, signatura, GPG, PGP, encryption, key, passphrase, contrasenya, gestor, password manager, Bitwarden, KeePass, 1Password, automàtica, cron, systemd, timer, scheduler, monitoratge, alerting, Uptime Kuma, alerta, Telegram, exit code, log, error, recovery, prova, test, dry-run, simulate, scenario, playbook, runbook.

Capitol 46 — 2FA, secrets i gestió de claus

*""Les contrasenyes soles ja no protegeixen res. El 2FA és el nou normal.""**

46.1 Per què el 2FA

Les contrasenyes tenen un problema: es poden endevinar, filtrar, o robar. Un cop algú té la teva contrasenya, té accés. El **2FA** (Two-Factor Authentication, autenticació de dos factors) afegeix un segon factor que **no es pot robar només amb la contrasenya**.

Els tres factors possibles:

1. **Quelcom que saps** (contrasenya, PIN).
2. **Quelcom que tens** (mòbil, clau USB).
3. **Quelcom que ets** (empremta, cara, veu).

El 2FA combina dos d'aquests, normalment **saps + tens**.

46.2 Tipus de 2FA

TOTP (Time-based One-Time Password)

El mòbil genera codis de 6 dígit que canvien cada 30 segons. Aplicacions: **Google Authenticator**, **Authy**, **1Password**, **Bitwarden**.

Exemple: 123 456 (canvia cada 30s).

Push notifications

Una app del mòbil (Duo, Microsoft Authenticator) t'avisa: "Vols iniciar sessió a BernatLab?" Tu acceptes.

SMS

Rebs un codi per SMS. **Molt poc segur** (SIM swapping), però millor que res.

Hardware tokens (claus físiques)

YubiKey, **Titan Key**, **OnlyKey**. Són claus USB que insertes al port. Molt segures.

Passkeys (WebAuthn / FIDO2)

L'estàndard modern. El mòbil o la clau USB fan d'identitat, sense contrasenya. Adoptat per Apple, Google, Microsoft.

Recomanació: **TOTP per compatibilitat** (molt acceptat) + **passkey o YubiKey** on estigui disponible.

46.3 Aplicar 2FA al BernatLab

2FA a Tailscale

Tailscale admet 2FA per accedir a la consola d'administració:

1. Vés a <https://login.tailscale.com/admin/settings/team>.
2. A "Two-factor authentication", configura TOTP.
3. Escaneja el codi QR amb Google Authenticator.
4. Desa els codis de recuperació en un lloc segur.

Cada vegada que accedeixis a la consola, et demanarà un codi.

2FA a Portainer

Portainer admet 2FA per a usuaris:

1. Crea un usuari a Portainer.
2. Activa 2FA a la configuració.
3. L'usuari escaneja el QR.

2FA a Grafana

Grafana admet 2FA:

1. A la configuració de l'usuari, activa 2FA.
2. Escaneja el QR.

2FA a Uptime Kuma

Uptime Kuma (versió 1.20+) admet 2FA. Configura'l igualment.

2FA a SSH

Per a SSH, pots usar:

1. **Claus SSH** (no és 2FA però és més segur que contrasenyes).
2. **OATH-TOTP** amb **libpam-google-authenticator**: a més de la clau, cal un codi TOTP.

Instal·lació:

```
sudo apt install libpam-google-authenticator
```

```
# A cada usuari  
google-authenticator
```

Configura `/etc/ssh/sshd_config`:

```
ChallengeResponseAuthentication yes  
AuthenticationMethods publickey,keyboard-interactive
```

I `/etc/pam.d/sshd`:

```
auth required pam_google_authenticator.so
```

Així, cal clau SSH + codi TOTP. Molt segur.

2FA a la Raspberry Pi (consola)

A la Raspberry, pots protegir l'inici de sessió local:

1. Instal·la `libpam-google-authenticator`.
2. Configura PAM per demanar TOTP a l'inici de sessió.

Això és útil si la Raspberry és accessible físicament (cosa que pot passar si la poses al camp).

46.4 Gestors de contrasenyes

El pitjor enemic de la seguretat són les **contrasenyes reutilitzades** i les **contrasenyes curtes**.

Solució: un **gestor de contrasenyes** que:

- Genera contrasenyes llargues i úniques.
- Emmagatzema tot xifrat.
- Autocompleta formularis.

Recomanats:

- **Bitwarden** (cloud, gratuït, multi-plataforma).
- **1Password** (pagament, excel·lent UX).
- **KeePassXC** (local, open source).
- **Apple Keychain** (integrat a macOS, bàsic però útil).

Per al BernatLab, **Bitwarden** és la millor opció:

- Gratuït per a ús personal.
- Multi-plataforma (Mac, Windows, Linux, mòbil).
- Open source (codi auditat).
- 2FA integrat.
- Compartició segura amb família.

46.5 Secrets al BernatLab

Secrets són credencials, claus d'API, contrasenyes, tokens. Al BernatLab tens molts:

- API keys (Homepage, Grafana, Portainer, InfluxDB).
- Contrasenyes de bases de dades.
- Tokens de Telegram bot.
- Claus Tailscale.
- AppEUI, AppKey dels nodes LoRa.
- API keys TTN (per a integració MQTT).

46.6 On guardar els secrets

■ Pitjor: en text pla al codi

```
# MAI NO!
TELEGRAM_TOKEN = "1234567890:ABCdefGHIjklMNOpqrsTUVwxyz"
```

Això és perillós perquè:

- Si el codi es penja a GitHub (fins i tot privat), queden exposats.
- Si la màquina es compromet, s'obtenen tots els secrets.

■■ Millor: variables d'entorn

```
import os

TELEGRAM_TOKEN = os.environ.get("TELEGRAM_TOKEN")
```

Les variables d'entorn **no es guarden al codi**, sinó a la configuració del sistema. Millor, però encara poden ser visibles al procés.

✓ Bo: fitxers .env (no versionats)

```
# .env (a .gitignore!)
TELEGRAM_TOKEN=1234567890:ABCdefGHIjklMNOpqrsTUVwxyz
```

Carrega-les amb `python-dotenv`:

```
from dotenv import load_dotenv
load_dotenv()

TELEGRAM_TOKEN = os.environ.get("TELEGRAM_TOKEN")
```

Afegeix `.env` a `.gitignore`.

✓✓ Molt bo: gestors de secrets

Per a sistemes seriosos:

- **HashiCorp Vault**: el més professional.
- **Bitwarden** (org): per a equips.
- **Doppler**: cloud, fàcil d'usar.
- **Infisical**: open source, cloud o self-hosted.
- **age** o **sops**: xifrar fitxers YAML/JSON.

✓✓✓ Excel·lent: Docker secrets

Si uses Docker Compose, pots usar **secrets**:

```
version: "3.8"
services:
  app:
    image: bernatlab/app
    secrets:
      - telegram_token

secrets:
```

```
telegram_token:
  file: ./secrets/telegram_token.txt
```

Els secrets es munten a `/run/secrets/` dins del contenidor. No apareixen a les variables d'entorn.

46.7 Rotació de secrets

Els secrets s'han de **rotar** periòdicament:

- **Contrasenyes d'usuari:** cada 6-12 mesos.
- **API keys:** cada 12 mesos o quan es sospita compromís.
- **Tokens:** cada 3-6 mesos.
- **Certificats:** cada 90 dies (Let's Encrypt ho fa automàticament).

Un calendari de rotació:

Secret	Freqüència	Responsable
Contrasenya Mac	6 mesos	Bernat
Contrasenya Raspberry	6 mesos	Bernat
Tailscale 2FA	quan cal	Bernat
Tokens Telegram	12 mesos	Bernat
API keys InfluxDB	12 mesos	Bernat
Clau SSH	12 mesos	Bernat
Certificat web	90 dies (auto)	Caddy/Traefik

46.8 Com generar contrasenyes segures

Una bona contrasenya:

- Té **mínim 16 caràcters**.
- Combina **majúscules, minúscules, números i símbols**.
- És **única** per a cada servei.
- **No es basa** en paraules de diccionari.

Exemple de bona contrasenya: `Tr0p1c@l-R@mb13r-2026!`

O més fàcil de recordar: `groc-tardor-poma-cullita-245m!` (4 paraules catalanes + número + signe).

Usa el gestor per generar-les:

- Bitwarden té un generador de contrasenyes.
- KeePassXC idem.
- 1Password idem.

46.9 Claus SSH

Les claus SSH són la millor manera d'autenticar-te a la Raspberry (molt millor que contrasenyes).

Generar una clau forta

```
ssh-keygen -t ed25519 -C "bernat@bernat-mbp" -f ~/.ssh/bernatlab
```

Això genera una clau ed25519 (més curta i segura que RSA).

Instal·lar-la a la Raspberry

```
ssh-copy-id -i ~/.ssh/bernatlab.pub bernat@hortosona
```

Això afegeix la clau pública a ~/.ssh/authorized_keys.

Desactivar l'accés per contrasenya

Edita /etc/ssh/sshd_config:

```
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
```

I reinicia SSH:

```
sudo systemctl restart sshd
```

Ara només pots accedir amb la clau. Molt més segur.

Protegir la clau privada amb passphrase

Quan generes la clau, et demana una passphrase. **Sempre posa'n una.** Així, si algú roba la clau, no pot fer-la servir.

Pots usar `ssh-agent` per no haver d'escriure la passphrase cada vegada:

```
ssh-add ~/.ssh/bernatlab
```

46.10 Gestió de claus a Tailscale

Tailscale té un sistema de **pre-auth keys** per afegir dispositius nous:

1. A la consola, genera una **pre-auth key**.
2. Fes servir aquesta clau al nou dispositiu: `tailscale up --authkey=tskey-....`
3. La clau té una durada limitada (per defecte, un sol ús).

També pots etiquetar automàticament:

```
tailscale up --authkey=tskey-... --tag=server
```

Així, quan s'uneix un dispositiu, ja té el tag correcte.

46.11 Pre-shared keys (PSK) per a serveis

Alguns serveis (WireGuard, OpenVPN, Wi-Fi WPA3) admeten **pre-shared keys** (PSK). Són claus compartides manualment que xifren el canal.

Exemple: WPA3-Personal usa una contrasenya com a PSK. Important que sigui forta.

46.12 Auditories periòdiques

Cada 3-6 mesos, revisa:

1. **Quins secrets** tens actius.
2. **Quins serveis** els usen.
3. **Si cal rotar** algun.
4. **Si hi ha secrets** antics que ja no es fan servir.
5. **Si hi ha secrets** exposats en algun lloc.

Eines útils:

- **git-secrets**: evita secrets al Git.
- **gitleaks**: cerca secrets en repositoris.
- **trufflehog**: cerca secrets en commits antics.

```
# Al repo del BernatLab
pip install gitleaks
gitleaks detect --source .
```

Si troba secrets, **rota'ls immediatament**.

46.13 Política de secrets al BernatLab

Documenta una política clara:

```
## Política de secrets al BernatLab

1. Tots els secrets s'emmagatzemen a `.env` (no versionat) o al gestor del sistema.
2. Les claus SSH tenen passphrase sempre.
3. 2FA activat a Tots els serveis que ho permetin.
4. Contrasenyes generades amb Bitwarden, mínim 16 caràcters.
5. Rotació de secrets cada 6-12 mesos.
6. Cap secret en text pla al codi o commits.
7. Auditoria amb gitleaks cada mes.
```

46.14 Errors habituals

Error 1: desar secrets a GitHub per error.

Assegura't que `.env` està a `.gitignore` abans del primer commit. Si ja l'has pujat, **rota els secrets immediatament** (no serveix de res esborrar el fitxer, el commit queda a la història).

Error 2: usar la mateixa contrasenya a tot arreu.

Si un servei es compromet, tots els altres cauen. Usa contrasenyes úniques.

Error 3: penjar la clau SSH a un sistema sense passphrase.

Si perds el portàtil, la clau queda exposada.

Error 4: desar secrets al README.

Els READMEs es comparteixen. Mai posis un secret visible al README.

Error 5: no rotar mai.

Els secrets vells són vectors d'atac. Rota'ls.

46.15 Resum

El 2FA, una bona gestió de contrasenyes, i una política clara de secrets són la tercera línia de defensa. TOTP i claus SSH amb passphrase són les millors pràctiques. Bitwarden és el millor gestor per a ús personal. Roteu els secrets periòdicament, i mai els poseu a Git. En el proper capítol veurem fail2ban, rate limiting, i tallafocs aplicat.

46.16 Exercicis pràctics

1. Instal·la Bitwarden (o un altre gestor) i migra les teves contrasenyes.
2. Activa 2FA a Tailscale, Portainer, Grafana, Uptime Kuma.
3. Genera claus SSH ed25519 amb passphrase.
4. Desactiva l'autenticació per contrasenya a la Raspberry.
5. Configura libpam-google-authenticator a la Raspberry per 2FA a SSH.
6. Crea un `.env` per a cada servei amb els secrets, i afegeix-lo a `.gitignore`.
7. Executa gitleaks al repo del BernatLab.
8. Documenta al README la política de secrets.

Paraules clau: **2FA, MFA, two-factor, autenticació, dos factors, TOTP, HOTP, Google Authenticator, Authy, 1Password, Bitwarden, passkey, FIDO2, WebAuthn, YubiKey, Titan, hardware token, clau física, SMS, push, push notification, OTP, one-time password, secret, password, contrasenya, passphrase, PIN, biometric, empremta, face ID, veu, iris, retina, smart card, targeta, NFC, RFID, password manager, gestor, vault, autofill, generador, password generator, random, secure random, entropy, PBKDF2, bcrypt, Argon2, scrypt, hash, salt, pepper, key stretching, strength, length, complexity, uniqueness, breach, leak, have i been pwned, HIBP, dictionary, attack, brute force, rainbow table, GPU, ASIC, cracking, rotation, expiry, lifetime, regen, rec, refresh, secret, API key, token, bearer, JWT, OAuth, OIDC, refresh token, access token, ssh-keygen, ed25519, RSA, ECDSA, public key, private key, keypair, fingerprint, authorized_keys, known_hosts, ssh-agent, ssh-add, ssh-copy-id, sshd, PasswordAuthentication, PermitRootLogin, ChallengeResponse, PAM, libpam-google-authenticator, OATH, RFC 6238, HOTP, security audit, gitleaks, git-secrets, trufflehog, detect-secrets, environment variable, .env, dotenv, secrets management, Vault, Infisical, Doppler, SOPS, age, gpg, encrypted file, KMS, HSM, cloud secret, AWS Secrets Manager, GCP Secret Manager, Azure Key Vault, Kubernetes secret, docker secret, compose secret, secret rotation, key rotation, certificate, TLS, x.509, ACME, Let's Encrypt, ZeroSSL, certbot, autorenew, expiry, CRL, OCSP, mTLS, mutual, client cert, server cert, root CA, intermediate CA, chain, trust store, openssl, verification, fingerprint, FQDN, SAN, Subject Alternative Name.**

Capitol 47 — fail2ban, rate limiting i tallafocs aplicat

"El millor tallafocs és el que t'avisa quan algú intenta passar."

47.1 Què és fail2ban

fail2ban és una eina que monitora els logs del sistema i **bloqueja IPs** que fan massa intents fallits. Per exemple, si una IP intenta accedir per SSH amb 5 contrasenyes errònies, fail2ban la bloqueja durant 10 minuts.

Combinat amb un tallafocs, és molt efectiu contra:

- **Bots d'escaneig** que busquen contrasenyes febles.
- **Atacs de força bruta** contra SSH, HTTP, FTP, etc.
- **Atacs de diccionari** contra serveis web.

47.2 Instal·lació a Debian/Raspberry

```
sudo apt update
sudo apt install fail2ban
```

Això instal·la fail2ban i l'inicia com a servei systemd.

Verificar:

```
sudo systemctl status fail2ban
```

47.3 Configuració bàsica

La configuració principal és a `/etc/fail2ban/jail.local` (crear-lo per no sobre escriure l'original):

```
[DEFAULT]
# Temps de bloqueig per defecte (1 hora)
bantime = 1h

# Finestra de temps per comptar intents fallits (10 min)
findtime = 10m

# Nombre d'intents abans del bloqueig
maxretry = 5

# IPs que mai no es bloquegen (loopback, xarxa local)
ignoreip = 127.0.0.1/8 100.64.0.0/10
```

El rang `100.64.0.0/10` és la xarxa Tailscale — important no bloquejar-te a tu mateix.

47.4 Jails (presons) actius

Cada "jail" és un patró de detecció. Els més útils:

SSH

```
[sshd]
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 24h
```

Així, 3 intents fallits = bloqueig durant 24 hores.

SSH amb Tailscale

Si només vols SSH des de Tailscale:

```
[sshd-tailscale]
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 24h
ignoreip = 100.64.0.0/10
```

I configura SSH per escoltar només a Tailscale:

```
sudo systemctl edit sshd
# A l'override.conf:
[Service]
ExecStart=
ExecStart=/usr/sbin/sshd -D -i -e -f /etc/ssh/sshd_config
```

I a `/etc/ssh/sshd_config`:

```
ListenAddress 100.115.134.76
```

HTTP (nginx)

```
[nginx-http-auth]
enabled = true
filter = nginx-http-auth
port = http,https
logpath = /var/log/nginx/error.log
maxretry = 3
bantime = 1h
```

Grafana, Portainer, etc.

Per a serveis web amb autenticació, pots crear jails personalitzats. Exemple per a Portainer:

```
[portainer-auth]
enabled = true
filter = portainer-auth
port = 9443
logpath = /var/lib/docker/volumes/portainer_data/_data/portainer.log
maxretry = 5
bantime = 1h
```

I el filtre a `/etc/fail2ban/filter.d/portainer-auth.conf`:

```
[Definition]
failregex = ^.*Failed login attempt for .* from <HOST>.*$
ignoreregex =
```

47.5 Comandes útils

Estat general

```
sudo fail2ban-client status
```

Estat d'un jail

```
sudo fail2ban-client status sshd
```

Veure IPs bloquejades

```
sudo fail2ban-client status sshd
# A la sortida:
# Banned IP list: 1.2.3.4, 5.6.7.8
```

Desbloquejar una IP

```
sudo fail2ban-client set sshd unbanip 1.2.3.4
```

Bloquejar una IP manualment

```
sudo fail2ban-client set sshd banip 1.2.3.4
```

Comprovar els logs

```
sudo tail -f /var/log/fail2ban.log
```

47.6 Com combinar amb el tallafocs

fail2ban usa **iptables** (o **nftables**) per bloquejar. Això és complementari al tallafocs normal.

Si tens UFW (Uncomplicated Firewall) activat:

```
# Comprovar si UFW està actiu
sudo ufw status

# Si no, activar
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow ssh
sudo ufw allow in on tailscale0 # permet tot des de Tailscale
sudo ufw enable
```

Ara UFW gestiona el tallafocs base, i fail2ban hi afegeix bloquejos dinàmics.

47.7 Rate limiting

El **rate limiting** limita el nombre de peticions per IP/segon. Això evita:

- **Atacs DoS** (denial of service).
- **Abús d'APIs** (excés de peticions).
- **Força bruta accelerada** (molts intents per segon).

Rate limiting a nginx

Exemple de limitació de peticions per IP:

```
http {
```

```
# Definir una zona de rate limiting
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;

server {
    location /api/ {
        limit_req zone=api burst=20 nodelay;
        proxy_pass http://backend;
    }
}
```

Això permet 10 peticions per segon, amb una ràfega de 20.

Rate limiting a Caddy

Caddy ho té integrat:

```
api.bernatlab.cat {
    rate_limit {remote.ip} 10r/s
    reverse_proxy localhost:8080
}
```

Rate limiting a Portainer

Portainer no té rate limiting natiu, però pots posar-lo darrere d'un reverse proxy (Caddy o nginx) que sí en tingui.

47.8 Protecció contra DoS

Per a una Raspberry, els atacs DoS són un risc real (la RPi no té gaire potència). Proteccions:

1. **Tailscale**: com que no exposa ports a Internet, ja està protegit.
2. **Cloudflare Tunnel**: si exposes serveis, Cloudflare filtra el tràfic.
3. **fail2ban**: bloqueja IPs sospitoses.
4. **Rate limiting**: limita peticions per IP.
5. **nginx amb limit_req**: filtra peticions abans d'arribar al backend.

47.9 Tallafocs aplicat a la Raspberry

Política per defecte: deny

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

Permetre només el necessari

```
# SSH només des de Tailscale
sudo ufw allow in on tailscale0 to any port 22

# Per a la gestió via web (si vols)
sudo ufw allow in on tailscale0 to any port 9443 # Portainer
sudo ufw allow in on tailscale0 to any port 3000 # Homepage
sudo ufw allow in on tailscale0 to any port 3001 # Uptime Kuma

# Activar
sudo ufw enable
sudo ufw status verbose
```

Limitar l'accés per IP

```
# Només tu pots accedir a SSH
sudo ufw allow from 100.115.134.76 to any port 22
```

47.10 Tallafocs al Mac

macOS té un tallafocs integrat (System Settings → Network → Firewall). Però és bàsic. Per a més control, pots usar **Lulu** o **Little Snitch** (gratuïts o de pagament).

Per configurar el tallafocs per aplicació:

1. System Settings → Network → Firewall → Options.
2. Activa "Block all incoming connections".
3. Permet les aplicacions que necessitin (Ollama, uvicorn, etc.).

47.11 Logs del tallafocs

Revisar els logs periòdicament:

```
# Logs de UFW
sudo tail -f /var/log/ufw.log

# Logs de fail2ban
sudo tail -f /var/log/fail2ban.log

# Logs d'auth (intents de SSH)
sudo tail -f /var/log/auth.log
```

Si veus molts intents des d'una IP estrangera, és un bot. fail2ban ja el bloquejarà, però val la pena investigar.

47.12 Protegir serveis exposats

Si per algun motiu exposes serveis a Internet (per exemple, amb Tailscale Funnel o Cloudflare Tunnel), calen proteccions addicionals:

1. **Cloudflare** filtra molt tràfic maliciós.
2. **Cloudflare Turnstile** o **hCaptcha** per a formularis.
3. **Rate limiting** agressiu.
4. **WAF** (Web Application Firewall, tallafocs d'aplicació web): Cloudflare, AWS WAF.
5. **Monitoratge** de peticions sospitoses.

47.13 Què fer quan et bloquegen a tu

Si t'has equivocat amb les ACLs o el tallafocs, pots quedar fora. Solucions:

1. **Accés físic** a la Raspberry: teclat + monitor.
2. **Portàtil amb Tailscale** des d'una altra xarxa.
3. **Una porta "break glass"** sempre oberta (però amb clau + 2FA).

4. Una VPN alternativa (WireGuard) com a backup.

Per a la Raspberry, el "break glass" és poder-hi accedir físicament amb un monitor. Configurar una sortida HDMI sempre.

47.14 Bones pràctiques

1. **Tallafocs per defecte deny.** Permet només el que cal.
2. **fail2ban** a tots els serveis amb autenticació.
3. **Rate limiting** a serveis web.
4. **Logs centralitzats** (Promtail, Loki, o un simple fitxer).
5. **Revisar setmanalment** els logs.
6. **Auditar mensualment** les regles del tallafocs.
7. **Documentar** cada canvi.

47.15 Resum

El tallafocs i fail2ban són la quarta línia de defensa. fail2ban bloqueja IPs que fan massa intents fallits. El tallafocs permet només el tràfic necessari. El rate limiting evita DoS. La combinació d'aquestes eines amb Tailscale (que ja evita l'exposició a Internet) fa que el BernatLab sigui molt segur. En el proper capítol veurem el hardening del sistema operatiu.

47.16 Exercicis pràctics

1. Instal·la fail2ban a la Raspberry.
2. Configura un jail per a SSH amb 3 intents màxim.
3. Activa UFW amb polítiques restrictives.
4. Configura rate limiting al reverse proxy.
5. Prova fail2ban: intenta 5 contrasenyes errònies per SSH.
6. Comprova les IPs bloquejades amb `fail2ban-client status sshd`.
7. Configura alertes a Telegram per a nous bloquejos.
8. Documenta al README l'estratègia de tallafocs.

Paraules clau: **fail2ban, jail, banned, banned IP, ban, unban, maxretry, findtime, bantime, ignoreip, recidive, jail.local, filter, action, iptables, nftables, UFW, uncomplicated firewall, iptables, nft, firewalld, pf, ipfw, ipfilter, IPFW, netfilter, Linux firewall, eBPF, Cilium, Falco, sysdig, audit, auditd, ausearch, aureport, logs, /var/log, /var/log/auth.log, /var/log/syslog, journald, journalctl, logrotate, centralitzat, Promtail, Loki, Grafana, alerting, Uptime Kuma, Telegram, rate limit, limit_req, limit_req_zone, burst, nodelay, leaky bucket, token bucket, fixed window, sliding window, counter, gauge, histogram, exponential backoff, DoS, DDoS, denial of service, distributed, amplification, reflection, SYN flood, UDP flood, ICMP flood, ping flood, ping of death, smurf, fraggle, teardrop, bonk, land, winnuke, nestea, jolt, OOB,**

out-of-band, RST, cookie, SYN cookie, tcp_syncookies, ip_no_pmtu_disc, rp_filter, reverse path filtering, martian, log_martians, accept_ra, accept_redirects, send_redirects, secure_redirects, proxy_arp, ip_forward, forwarding, route, gateway, default gateway, network namespace, netns, VRF, table, FIB, RIB, rules, chains, INPUT, OUTPUT, FORWARD, PREROUTING, POSTROUTING, nat, masquerade, SNAT, DNAT, REDIRECT, LOG, REJECT, DROP, ACCEPT, queue, NFQUEUE, conntrack, connection tracking, state, NEW, ESTABLISHED, RELATED, INVALID, UNTRACKED, rate, limit, policer, shaper, tc, traffic control, netem, qdisc, pfifo, fq_codel, CAKE, RED, ECN, explicit congestion notification, ECN, CE, ECT, ect_0, ect_1, ECT(0), ECT(1), Not-ECT, Not-ECT, Low Delay, Low Loss, Low Loss Low Delay, L4S, Prague, dual queue, fq, fair queue, FQ-CoDel, Stochastic Fair Queueing, SFQ, priority, qos, dscp, tos, vlan, 802.1Q, qinq, 802.1ad, bridge, brctl, veth, macvlan, ipvlan, tunnel, GRE, IPIP, GUE, VXLAN, GENEVE, ERSPAN, ERSPAN, Type, ERSPAN, ERSPAN Type II, ERSPAN Type III, ERSPAN Type I, header, payload, encap, decap, geneve, vni, vxlan, segment, ID, encapsulation, tunnel endpoint, VTEP, VTEP, VTEP, distributed, virtual, switch, Open vSwitch, OVS, DPDK, FD.io, VPP, fast data, fast path, kernel bypass, XDP, eXpress Data Path, AF_XDP, busy polling, epoll, io_uring, sendfile, splice, tee, vmsplice, copy avoidance, zero copy, kernel-bypass, user-space networking, networking, layer 2, layer 3, layer 4, layer 7, application, presentation, session, transport, network, data link, physical, MAC, PHY, switch, hub, bridge, router, gateway, firewall, IDS, IPS, NDR, network detection, response, DPI, deep packet inspection, NF, netfilter, iptables, nftables, ebttables, arptables, ip6tables, brctl, ip rule, ip route, ip neigh, ip tunnel, ip link, ip addr, ip maddr, ip mroute, ip xfrm, ip netns, ip l2tp, ip tcp_metrics, ss, netstat, lsof, fuser, lscpu, lspci, lsusb, lsblk, lsscsi, lsraid, dmidecode, lshw, hwdm, inxi, screenfetch, neofetch, fastfetch, weather, cpu, gpu, ram, rom, bios, uefi, legacy, efivars, secure boot, tpm, measured boot, attested, measured, root of trust, intel TXT, AMD SKINIT, SRTM, DRTM, late launch, dynamic root of trust, trust, attestation, measured, integrity, IMA, EVM, Linux IMA, Integrity Measurement Architecture, Extended Verification Module, appraisal, audit, signed, kernel, signed, modules, signed, dm-verity, dm-integrity, LUKS, LUKS2, cryptsetup, LUKS header, key slot, passphrase, TPM, enrollment, key, binding, PCR, Platform Configuration Register, policy, TPM2, TPM2_CreatePrimary, TPM2_StartAuthSession, TPM2_PolicyPCR, TPM2_PCR_Read, TPM2_NV_Read, TPM2_NV_DefineSpace, TPM2_NV_Write, TPM2_NV_ReadLock, TPM2_NV_WriteLock, TPM2_Clear, TPM2_DictionaryAttackLockReset, TPM2_DictionaryAttackParameters, TPM2_Startup, TPM2_Shutdown, TPM2_GetRandom, TPM2_GetTestResult, TPM2_GetCapability, TPM2_ReadPublic, TPM2_Import, TPM2_Load, TPM2_LoadExternal, TPM2_Seal, TPM2_Unseal, TPM2_Sign, TPM2_VerifySignature, TPM2_RSA_Decrypt, TPM2_ECC_ZGen, TPM2_ECC_Sign, TPM2_ECC_Verify, TPM2_ZGen_2Phase, TPM2_ECC_Point_Z, TPM2_Encrypt, TPM2_Decrypt, TPM2_HMAC, TPM2_HMAC_Start, TPM2_HMAC_Update, TPM2_HMAC_Complete, TPM2_SequenceComplete, TPM2_SequenceUpdate, TPM2_Sign, TPM2_Verify, TPM2_Certify, TPM2_CertifyCreation, TPM2_Quote, TPM2_PCR_Allocate, TPM2_PCR_SetAuthPolicy, TPM2_PCR_SetAuthValue, TPM2_NV_Certify, TPM2_PCR_Reset, TPM2_ChangeAuth, TPM2_NV_ChangeAuth, TPM2_LoadKey, TPM2_Seal, TPM2_Unseal, TPM2_Duplicate, TPM2_Rewrap, TPM2_Import, TPM2_LoadExternal, TPM2_LoadKey, TPM2_LoadKey2, TPM2_ReadPublic, TPM2_RSA_Encrypt, TPM2_RSA_Decrypt, TPM2_ECDH_KeyGen, TPM2_ECDH_ZGen, TPM2_ECC_Parameters, TPM2_FirmwareRead, TPM2_Capability, TPM2_GetRandom, TPM2_GetTestResult, TPM2_Increment, TPM2_ReadClock, TPM2_ReadClock, TPM2_TickStamp, TPM2_Time, TPM2_Verify, TPM2_NV_GlobalWriteLock,

TPM2_NV_Increment, TPM2_NV_Extend, TPM2_Vendor_TCG, vendor, defined, commands.

Capítol 48 — Hardening del sistema operatiu

"El sistema operatiu és la base de tot. Si la base és feble, tot el que hi construeixes és feble."

48.1 Què és el hardening

Hardening és el procés de fer un sistema operatiu **més segur** reduint la superfície d'atac. Això inclou:

- Desactivar serveis innecessaris.
- Aplicar pegats de seguretat.
- Configurar permisos estrictes.
- Activar proteccions del kernel.
- Limitar capacitats dels processos.

48.2 Les 4 fases del hardening

1. **Inventari**: què tens instal·lat, què s'executa, quins ports estan oberts.
2. **Reducció**: elimina tot el que no necessitis.
3. **Configuració**: aplica configuracions segures.
4. **Verificació**: comprova que tot funciona i està protegit.

48.3 Inventari inicial

Abans de fer canvis, cal saber què tens:

Quins serveis s'executen

```
sudo systemctl list-units --type=service --state=running
```

Quins ports escolten

```
sudo ss -tulpn
```

Quins paquets estan instal·lats

```
sudo apt list --installed
```

Quins usuaris existeixen

```
cat /etc/passwd | grep -v nologin | grep -v false
```

Quins grups tenen membres

```
sudo getent group sudo admin docker
```

48.4 Actualitzacions

El primer pas, sempre: **mantenir el sistema actualitzat**.

```
# Actualitza la llista
sudo apt update

# Mostra què hi ha pendent
sudo apt list --upgradable

# Actualitza tot
sudo apt upgrade

# Actualitza amb canvis de kernel
sudo apt full-upgrade

# Neteja paquets obsolets
sudo apt autoremove
```

Actualitzacions automàtiques

Per no oblidar-te'n:

```
sudo apt install unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
```

Edita `/etc/apt/apt.conf.d/50unattended-upgrades`:

```
Unattended-Upgrade::Allowed-Origins {
    "Debian bookworm-security";
    "Debian bookworm-updates";
};
Unattended-Upgrade::AutoFixInterruptedDpkg "true";
Unattended-Upgrade::MinimalSteps "true";
Unattended-Upgrade::Remove-Unused-Kernel-Packages "true";
Unattended-Upgrade::Remove-Unused-Dependencies "true";
Unattended-Upgrade::Automatic-Reboot "false";
```

Ara el sistema s'actualitza sol cada dia.

48.5 Desactivar serveis innecessaris

Cada servei actiu és una porta potencial. Desactiva tot el que no necessitis.

A la Raspberry, serveis típics que pots desactivar:

```
sudo systemctl disable --now avahi-daemon # mDNS
sudo systemctl disable --now bluetooth    # si no uses
sudo systemctl disable --now cups         # impressió
sudo systemctl disable --now ModemManager # mòdem
sudo systemctl disable --now wpa_supplicant # si no uses Wi-Fi
```

Per veure què consumeix recursos:

```
sudo systemctl list-units --type=service --state=running --no-pager
```

48.6 Proteccions del kernel (sysctl)

`sysctl` permet configurar paràmetres del kernel en temps d'execució. Crea `/etc/sysctl.d/99-security.conf`:

```
# Protecció contra IP spoofing
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1

# No acceptar redireccions ICMP
net.ipv4.conf.all.accept_redirects = 0
```

```

net.ipv4.conf.default.accept_redirects = 0
net.ipv6.conf.all.accept_redirects = 0
net.ipv6.conf.default.accept_redirects = 0

# No enviar redireccions
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.default.send_redirects = 0

# No acceptar IP source routing
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.conf.default.accept_source_route = 0
net.ipv6.conf.all.accept_source_route = 0
net.ipv6.conf.default.accept_source_route = 0

# Habilitar SYN cookies (protecció contra SYN flood)
net.ipv4.tcp_syncookies = 1

# No habilitar IP forwarding (si no ets router)
net.ipv4.ip_forward = 0
net.ipv6.conf.all.forwarding = 0

# Protecció contra martians
net.ipv4.conf.all.log_martians = 1

# Kernel: randomize memory layout (ASLR)
kernel.randomize_va_space = 2

# No exposar informació del kernel
kernel.dmesg_restrict = 1
kernel.kptr_restrict = 2

# Restringir accés a /proc
fs.protected_hardlinks = 1
fs.protected_symlinks = 1

```

Aplica:

```
sudo sysctl -p /etc/sysctl.d/99-security.conf
```

48.7 Permissos dels fitxers crítics

Molts fitxers del sistema tenen permisos massa oberts. Assegura't que estiguin correctes:

```

# Permissos estrictes
sudo chmod 600 /etc/shadow
sudo chmod 600 /etc/gshadow
sudo chmod 644 /etc/passwd
sudo chmod 644 /etc/group
sudo chmod 600 /etc/ssh/ssh_config
sudo chmod 600 /boot/grub/grub.cfg # si tens GRUB

# Propietaris
sudo chown root:root /etc/shadow /etc/gshadow /etc/passwd /etc/group

```

48.8 SSH hardening

L'SSH és una porta d'entrada crítica. Configuració segura a `/etc/ssh/sshd_config`:

```

# Port personalitzat (no és segur, però redueix bots)
Port 22

# Protocol modern
Protocol 2

```

```
# Desactivar root login
PermitRootLogin no

# Només autenticació per clau
PasswordAuthentication no
PubkeyAuthentication yes
PermitEmptyPasswords no

# Limitar usuaris
AllowUsers bernat

# Limitar intents
MaxAuthTries 3
MaxSessions 3

# Desactivar tunneling innecessari
AllowTcpForwarding no
AllowAgentForwarding no
X11Forwarding no

# Log verbós
LogLevel VERBOSE

# Banner d'avís
Banner /etc/issue.net
```

[A /etc/issue.net](#):

ALERTA: Aquest sistema és privat. Accés no autoritzat prohibit.
Totes les connexions queden registrades.

Reiniciar SSH:

```
sudo systemctl restart sshd
```

48.9 AppArmor o SELinux

AppArmor (Debian/Ubuntu) i **SELinux** (RHEL/Fedora) són sistemes de **control d'accés obligatori** (MAC, Mandatory Access Control). Confinen processos perquè només puguin fer el que han de fer.

Per al BernatLab, **AppArmor** és la millor opció:

```
# Comprovar si està actiu
sudo aa-status

# Instal·lar perfils
sudo apt install apparmor apparmor-utils apparmor-profiles
```

Activar per defecte:

```
sudo systemctl enable apparmor
sudo systemctl start apparmor
```

Forçar tots els perfils:

```
sudo aa-enforce /etc/apparmor.d/*
```

Els perfils estan a [/etc/apparmor.d/](#). Cada perfil defineix què pot fer un programa (quins fitxers pot llegir, quines xarxes pot usar, etc.).

Exemple de perfil per a un servei:

```
#include <tunables/global>
```

```
/usr/bin/mosquitto {
    #include <abstractions/base>
    #include <abstractions/nameservice>

    capability net_bind_service,
    capability setgid,
    capability setuid,

    /etc/mosquitto/mosquitto.conf r,
    /etc/mosquitto/conf.d/ r,
    /etc/mosquitto/conf.d/* r,
    /etc/mosquitto/passwordfile r,
    /var/lib/mosquitto/ r,
    /var/lib/mosquitto/* rwk,
    /var/log/mosquitto/ r,
    /var/log/mosquitto/* w,

    network tcp,
    network udp,
}
```

48.10 Desactivar USB automàtic

Si la Raspberry és accessible, algú podria connectar un USB maliciós. Desactiva el muntatge automàtic:

```
sudo apt remove udisks2
```

O amb regla udev:

```
# /etc/udev/rules.d/01-block-usb.conf
ACTION=="add", SUBSYSTEMS=="usb", ENV{UDISKS_IGNORE}="1"
```

48.11 BIOS/UEFI password

Si la teva màquina té BIOS/UEFI, posa-hi contrasenya. Això evita que algú pugui canviar la configuració d'arrencada.

A la Raspberry, no cal (no té BIOS configurable), però al Mac sí:

1. Reinicia el Mac.
2. Manté **Cmd+R** per entrar a recovery.
3. Utilitats → Utilitat de Contrasenyes del Firmware.
4. Activa la contrasenya.

48.12 Protecció física

La seguretat física importa:

- La Raspberry ha d'estar en un lloc **tancat** (armari amb clau).
- El Mac ha d'estar en un lloc **segur** (no accessible a convidats).
- Els dispositius extraïbles (USB) han d'estar **guardats**.
- Les còpies externes han d'estar en un **lloc diferent**.

48.13 sysctl i el rendiment

Algunes configuracions sysctl poden afectar el rendiment:

- `vm.swappiness = 10` (evita intercanvi excessiu, recomanat per a servidors).
- `net.core.somaxconn = 1024` (més connexions entrants).
- `net.ipv4.tcp_tw_reuse = 1` (reutilitza connexions TIME_WAIT).

Per al BernatLab, el rendiment no és crític (pocs usuaris), però no va malament.

48.14 Netdata: monitoratge bàsic

Per veure què passa al sistema, instal·la **Netdata**:

```
# A la Raspberry  
sudo apt install netdata
```

Això t'ofereix un panell web a `http://localhost:19999` amb gràfiques en temps real de:

- CPU, RAM, disc, xarxa.
- Per procés.
- Per servei.

Molt útil per entendre el sistema.

48.15 Auditories automàtiques amb Lynis

Lynis és una eina d'auditoria de seguretat per a Linux. Escaneja el sistema i proposa millores:

```
sudo apt install lynis  
sudo lynis audit system
```

Lynis revisa centenars de controls i et dona un informe amb advertències i suggeriments. Executa'l mensualment.

48.16 Checklist de hardening

Un resum del que cal fer:

- ☐ Actualitzar el sistema regularment.
- ☐ Desactivar serveis innecessaris.
- ☐ Configurar sysctl per a proteccions del kernel.
- ☐ Permisos estrictes als fitxers crítics.
- ☐ SSH: només clau, no root, ports limitats.
- ☐ 2FA a SSH (libpam-google-authenticator).
- ☐ AppArmor actiu amb perfils.
- ☐ Tallafocs configurat (UFW).
- ☐ fail2ban actiu.

- [] Logs centralitzats i revisats.
- [] Lynis o eina similar cada mes.
- [] Documentació de tot al README.

48.17 Resum

El hardening és un procés continu: inventari, reducció, configuració, verificació. Les accions clau són actualitzar, desactivar serveis, configurar el kernel amb sysctl, endurir SSH, activar AppArmor, i auditar regularment amb Lynis. Al proper capítol veurem com auditar, monitorar logs de seguretat, i respondre a incidents.

48.18 Exercicis pràctics

1. Inventaria serveis, ports, paquets i usuaris de la Raspberry.
2. Configura `unattended-upgrades` per a actualitzacions automàtiques.
3. Desactiva 3 serveis innecessaris.
4. Aplica la configuració sysctl de seguretat.
5. Endureix la configuració SSH.
6. Activa AppArmor amb perfils.
7. Instal·la Lynis i executa'l.
8. Documenta al README l'estat de hardening.

Paraules clau: **hardening**, **endurement**, **superfície d'atac**, **attack surface**, **sysctl**, **kernel**, **paràmetres**, **rp_filter**, **martian**, **redirect**, **source route**, **SYN cookies**, **ASLR**, **kptr_restrict**, **dmesg_restrict**, **fs.protected_hardlinks**, **fs.protected_symlinks**, **ip_forward**, **accept_ra**, **AppArmor**, **SELinux**, **MAC**, **mandatory access control**, **perfil**, **profile**, **enforce**, **complain**, **audit**, **aa-genprof**, **aa-logprof**, **apparmor_parser**, **aa-status**, **aa-enforce**, **aa-complain**, **abstractions**, **capability**, **file**, **network**, **mount**, **ptrace**, **signal**, **capability**, **capabilities**, **Linux capabilities**, **CAP_NET_BIND_SERVICE**, **CAP_SYS_ADMIN**, **CAP_DAC_OVERRIDE**, **CAP_CHOWN**, **CAP_FOWNER**, **CAP_FSETID**, **CAP_KILL**, **CAP_SETGID**, **CAP_SETUID**, **CAP_SETPCAP**, **CAP_LINUX_IMMUTABLE**, **CAP_NET_BIND_SERVICE**, **CAP_NET_BROADCAST**, **CAP_NET_ADMIN**, **CAP_NET_RAW**, **CAP_IPC_LOCK**, **CAP_IPC_OWNER**, **CAP_SYS_MODULE**, **CAP_SYS_RAWIO**, **CAP_SYS_CHROOT**, **CAP_SYS_PTRACE**, **CAP_SYS_PACCT**, **CAP_SYS_ADMIN**, **CAP_SYS_BOOT**, **CAP_SYS_NICE**, **CAP_SYS_RESOURCE**, **CAP_SYS_TIME**, **CAP_SYS_TTY_CONFIG**, **CAP_MKNOD**, **CAP_LEASE**, **CAP_AUDIT_WRITE**, **CAP_AUDIT_CONTROL**, **CAP_SETFCAP**, **CAP_MAC_OVERRIDE**, **MAC**, **mandatory access control**, **SELinux**, **AppArmor**, **TOMOYO**, **Yama**, **seccomp**, **seccomp-bpf**, **seccomp-filter**, **filter**, **sandbox**, **prctl**, **PR_SET_NO_NEW_PRIVS**, **PR_SET_SECCOMP**, **namespace**, **mount namespace**, **PID namespace**, **network namespace**, **user namespace**, **cgroup namespace**, **cgroup**, **control group**, **cgroup v1**, **cgroup v2**, **systemd-cgroup**, **slice**, **scope**, **service**, **unit**, **unit file**, **drop-in**, **override**, **sysctl**, **/etc/sysctl.d/**, **/proc/sys/**, **/proc/sys/net/**, **/proc/sys/kernel/**, **/proc/sys/fs/**, **mount**, **/etc/fstab**, **options**, **nodev**, **nosuid**, **noexec**, **ro**, **relatime**, **noatime**, **securetty**, **/etc/securetty**, **login**, **faillog**, **faillock**, **pam**, **faillock**, **pam_faillock**, **pam_tally2**,

pam_cracklib, pam_pwquality, password quality, complexity, length, history, reuse, dictionary, cracklib, libpam-pwquality, pwquality.conf, /etc/security/pwquality.conf, minlen, dcredit, ucredit, lcredit, ocredit, difok, dictpath, enforce_for_root, retry, remember, sha512, rounds, pam_unix, pam_pwquality, pam_passwdqc, libpam-pkcs11, smart card, certificate, PIV, CAC, Yubikey, U2F, FIDO2, libpam-u2f, libpam-fido2, libpam-yubikey, Yubico PAM, Yubikey, hardware token, libpam-google-authenticator, OATH, TOTP, RFC 6238, RFC 4226, QR code, otpauth, secret, key URI, otpauth-migration, otp, htop, etc, audit, auditd, auditctl, ausearch, aureport, rules, /etc/audit/rules.d/, watch, syscall, file watch, syscall audit, network audit, execve, fork, connect, accept, bind, listen, open, read, write, close, change, profile, snapshot, report, malware, rootkit, chkrootkit, rkhunter, clamav, clamscan, freshclam, signature, update, scan, alert, notification, AIDE, tripwire, integrity, file integrity, baseline, hash, SHA-256, SHA-512, signature, GPG, signed manifest, Tripwire, OSSEC, Wazuh, ELK, Elasticsearch, Logstash, Kibana, SIEM, log management, retention, archive, tier, search, query, dashboard, alert, contact, template, channel, recipient, trigger, threshold, condition, severity, action, suppression, deduplication, correlation, alert, playbook, runbook, incident, response, NIST, SANS, CREST, OWASP, top 10, CWE, CVE, CVSS, EPSS, KEV, catalog, known exploited, vulnerability, scanner, OpenVAS, Nessus, Qualys, Rapid7, InsightVM, Tenable, cloud, security, posture, CSPM, CWPP, CIEM, IAM, least privilege, Zero Trust, ZTNA, SDP, software defined perimeter, BeyondCorp, BeyondCorp Enterprise, identity-aware proxy, IAP, access proxy, OAuth, OIDC, SAML, JWT, mTLS, certificate, x.509, SPIFFE, SPIRE, identity, attestation, workload, mTLS, mesh, service mesh, Istio, Linkerd, Consul, mTLS, automatic, sidecar, ambient, mesh, multi-cluster, multi-region, federation, multi-cloud, hybrid, multitenancy, tenant, isolation, namespace, policy, authorization, OPA, Open Policy Agent, Rego, Cedar, decision, audit, decision log.

Capítol 49 — Auditoria, logs de seguretat i resposta a incidents

"Els logs són la caixa negra del teu sistema. Si no els mires, no tens caixa negra."

49.1 Què és la auditoria de seguretat

Auditoria és el procés de revisar periòdicament el sistema per:

- Detectar accessos no autoritzats.
- Identificar configuracions errònies.
- Trobar patrons sospitosos.
- Complir normatives (si escau).

Una bona auditoria és **proactiva**: detecta problemes abans que es converteixin en incidents.

49.2 Logs essencials

Al BernatLab, els logs clau són:

Logs de sistema

- `/var/log/syslog`: missatges generals del sistema.
- `/var/log/auth.log`: autenticació (intents de SSH, sudo).
- `/var/log/kern.log`: missatges del kernel.
- `/var/log/dpkg.log`: instal·lació de paquets.
- `/var/log/cron.log`: tasques programades.

Logs d'aplicacions

- **Docker**: `docker logs <container>`.
- **Grafana**: `/var/lib/docker/volumes/grafana-data/_data/log/`.
- **InfluxDB**: `docker logs influxdb`.
- **fail2ban**: `/var/log/fail2ban.log`.
- **Uptime Kuma**: interfície web.
- **Tailscale**: `tailscale status`, `tailscale netcheck`.

49.3 Comandes bàsiques de logs

journalctl (systemd)

```
# Tots els logs d'avui
journalctl --since today

# Errors
journalctl -p err

# Per un servei específic
journalctl -u sshd

# En temps real
journalctl -f

# Des d'una data
journalctl --since "2026-07-08 10:00"
```

grep i awk

```
# Tots els accessos SSH fallits
grep "Failed password" /var/log/auth.log

# Tots els accessos SSH amb èxit
grep "Accepted" /var/log/auth.log

# IPs que han intentat accedir
awk '/Failed password/ {print $(NF-3)}' /var/log/auth.log | sort -u

# Errors de Docker
docker logs --tail 100 bernatlab_grafana_1
```

49.4 Què buscar als logs

Senyals d'alerta:

1. **Molts intents d'autenticació fallits** des d'una IP.
2. **Accesos exitosos** des d'IPs estranyes.
3. **Canvis en fitxers crítics** (`/etc/passwd`, `/etc/shadow`, `/etc/ssh/`).
4. **Serveis nous** que s'han iniciat sense motiu.
5. **Tràfic de xarxa anormal** (especialment cap a IPs externes).
6. **Ús elevat de CPU/RAM** en horaris estranys.
7. **Comandes sospitoses** a l'historial de bash.
8. **Fitxers nous** a directoris sensibles (`/tmp`, `/var/tmp`).

49.5 Logs centralitzats

Quan tens múltiples dispositius, els logs centralitzats són clau. Opcions:

Loki + Grafana

Loki és un sistema de logs de Grafana Labs, lleuger i fàcil d'integrar.

Instal·lació a la Raspberry:

```
# Loki
docker run -d --name loki \
  -v /home/bernat/homelab/loki:/etc/loki \
  -p 3100:3100 \
  grafana/loki:2.9.0

# Promtail (agent que envia logs)
docker run -d --name promtail \
  -v /var/log:/var/log \
  -v /home/bernat/homelab/promtail:/etc/promtail \
  grafana/promtail:2.9.0
```

Configuració de Promtail (/etc/promtail/config.yml):

```
server:
  http_listen_port: 9080

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: system
    static_configs:
      - targets:
          - localhost
        labels:
          job: syslog
          __path__: /var/log/*.log

  - job_name: auth
    static_configs:
      - targets:
          - localhost
        labels:
          job: auth
          __path__: /var/log/auth.log
```

A Grafana, afegeix Loki com a data source i pots fer cerques com:

```
{job="auth"} |= "Failed password"
```

Alternatives

- **Graylog**: més pesat, però complet.
- **ELK Stack** (Elasticsearch + Logstash + Kibana): el més potent, però molt pesat.
- **Netdata**: ja l'hem vist al Cap 48.
- **Prometheus + Grafana**: per a mètriques (no logs), però útil per alertes.

49.6 Alertes de seguretat

Quan els logs mostren alguna cosa sospitosa, cal **actuar ràpid**. Les alertes automatitzades són la millor eina.

Alertes amb Grafana + Loki

```
# Alerta: més de 10 intents SSH fallits en 5 minuts
- alert: SSHBruteForce
  expr: |
    sum by (remote_addr) (
      rate({job="auth"} |= "Failed password" [5m])
    ) > 0.1
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "Possible atac de força bruta SSH des de {{ $labels.remote_addr }}"
    description: "Més de 10 intents fallits en 5 minuts. IP bloquejada per fail2ban."
```

Alertes amb Uptime Kuma

Uptime Kuma pot monitorar serveis i alertar via Telegram, correu, etc.

Alertes amb scripts personalitzats

Un script simple que revisa logs i alerta per Telegram:

```
#!/bin/bash
# check_security.sh

# Comprovar intents SSH fallits en l'última hora
FAILED=$(grep "Failed password" /var/log/auth.log | grep "$(date '+%b %e %H' -d '1 hour ago')" | wc -l)

if [ $FAILED -gt 20 ]; then
    curl -X POST "https://api.telegram.org/bot$TELEGRAM_TOKEN/sendMessage" \
        -d "chat_id=$CHAT_ID" \
        -d "text=■■ Alerta BernatLab: $FAILED intents SSH fallits en 1h!"
fi

# Comprovar canvis a fitxers crítics
find /etc -name "*.conf" -mtime -1 2>/dev/null | while read f; do
    curl -X POST "https://api.telegram.org/bot$TELEGRAM_TOKEN/sendMessage" \
        -d "chat_id=$CHAT_ID" \
        -d "text=■ Fitxer modificat: $f"
done
```

Afegeix a cron cada hora.

49.7 IDS (Intrusion Detection System)

Un **IDS** detecta intrusions comparant el comportament amb patrons coneguts.

AIDE (Advanced Intrusion Detection Environment)

AIDE revisa canvis en fitxers del sistema:

```
sudo apt install aide

# Inicialitzar la base de dades
sudo aideinit
# Això crea /var/lib/aide/aide.db.new

# Comparar amb l'estat actual
sudo aide.wrapper --check

# Actualitzar la base
```

```
sudo cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db
```

Configurar a `/etc/aide/aide.conf`:

```
# Directoris a monitorar
/boot Full
/bin Full
/sbin Full
/etc Full
/usr Full
/var/log Full
```

AIDE revisa atributs, mides, hash, etc. Si algun canvia, t'avisarà.

OSSEC / Wazuh

Wazuh (abans OSSEC) és un IDS complet amb:

- Monitor de fitxers.
- Detector d'intrusions.
- Anàlisi de logs.
- Alertes.

Per al BernatLab, és overkill, però útil si vols auditar molt.

49.8 Resposta a incidents

Quan es detecta un incident, cal un **procediment** clar:

Les 6 fases de la resposta

1. **Preparació.** Tenim eines, runbooks, contactes.
2. **Detecció.** Hem vist alguna cosa sospitosa.
3. **Anàlisi.** Què ha passat exactament?
4. **Contenció.** Aturar l'atac.
5. **Erradicació.** Treure l'atacant del sistema.
6. **Recuperació.** Tornar a la normalitat.
7. **Post-incident.** Aprendre i millorar.

Runbook per a una bretxa

Un **runbook** és un procediment pas a pas. Exemple:

```
# Runbook: bretxa de seguretat al BernatLab

## Detecció
- Alerta de Uptime Kuma.
- Logs de fail2ban.
- Canvis a fitxers crítics.

## Pas 1: Contenció immediata
1. Desconnectar la Raspberry de Tailscale.
2. Apagar serveis sospitosos.
```


3. Canviar TOTES les contrasenyes.
4. Revocar TOTES les claus API.

Pas 2: Anàlisi

1. Revisar logs d'autenticació.
2. Buscar fitxers nous a /tmp, /var/tmp.
3. Comprovar cron jobs nous.
4. Mirar tràfic de xarxa amb `ss` o `tcpdump`.

Pas 3: Erradicació

1. Identificar el vector d'entrada.
2. Tancar-lo.
3. Esborrar qualsevol malware.
4. Restaurar des de còpies netes.

Pas 4: Recuperació

1. Tornar a engegar serveis.
2. Verificar que funcionen correctament.
3. Monitorar intensivament les primeres 24h.

Pas 5: Post-incident

1. Documentar què ha passat.
2. Identificar millores.
3. Aplicar-les.
4. Comunicar a les parts afectades.

49.9 Comunicació d'incidents

Si hi ha dades personals afectades, cal:

1. **Notificar els afectats** (tu, en aquest cas).
2. **Si escau, notificar l'APDCAT** (Autoritat Catalana de Protecció de Dades).
3. **Si escau, notificar els Mossos o la Policia Nacional** (si és delictes).
4. **Documentar tot** per a l'auditoria.

49.10 Evidències forenses

Si vols investigar a fons, cal preservar evidències:

1. **No apagar el sistema** (perdries memòria volàtil).
2. **Fer còpia del disc** amb `dd` o `ddrescue`.
3. **Capturar memòria** amb `avml` o `LiME`.
4. **Copiar logs** abans que roten.
5. **Documentar l'ordre** de les accions.

Eines útils:

- **The Sleuth Kit (TSK)**: anàlisi forense de disc.
- **Volatility**: anàlisi de memòria.
- **Autopsy**: interfície gràfica per a TSK.

49.11 Auditories periòdiques

Un calendari d'auditoria recomanat:

Freqüència	Què auditar
Diàriament	Alertes de Uptime Kuma, intents SSH.
Setmanalment	Logs d'auditoria, canvis de paquets.
Mensualment	Lynis, AIDE, revisió d'usuaris.
Trimestralment	Auditoria completa, còpies, runbooks.
Anualment	Penetració testing (si tens temps), disaster recovery drill.

49.12 Errors habituals

Error 1: no mirar mai els logs.

Si no mires els logs, els incidents passen sense que te n'adonis. Mira'ls almenys un cop per setmana.

Error 2: confiar en la seguretat per obscuritat.

"Ostres, amago el port SSH al 1234" no és seguretat. Només redueix els bots automatitzats.

Error 3: no tenir cap runbook.

Quan passa un incident, el pànic et porta a cometre errors. Un runbook t'ajuda a actuar correctament.

Error 4: no comunicar l'incident.

Per vergonya o por, alguns ho amaguen. Això és pitjor. Comunica, aprèn, millora.

Error 5: sobre-reaccionar.

Si veus un sol intent fallit, no cal apagar tota la infraestructura. Mantén la calma, avalua, respon.

49.13 Resum

L'auditoria periòdica i el monitoratge de logs són la cinquena línia de defensa. Logs centralitzats amb Loki + Grafana, alertes automatitzades, IDS com AIDE, i runbooks clars per a la resposta a incidents. La preparació és la clau: quan passa alguna cosa, has de saber què fer sense pensar-ho. En el proper capítol veurem el pla de recuperació davant desastres (DRP), l'última capa de defensa.

49.14 Exercicis pràctics

1. Configura Loki + Promtail per centralitzar logs.
2. Crea una alerta a Grafana per a intents SSH fallits.
3. Configura AIDE per monitorar fitxers crítics.
4. Escriu un runbook per a una bretxa de seguretat.

5. Executa Lynis i revisa les advertències.
6. Configura alertes per Telegram via un script.
7. Documenta al README l'estratègia de monitoratge.

Paraules clau: auditoria, audit, seguretat, security, monitoring, alertes, alerting, detecció, response, incident, runbook, playbook, NIST, SANS, 6 fases, preparació, anàlisi, contenció, erradicació, recuperació, post-incident, post-mortem, blameless, AAR, after action review, lessons learned, millora contínua, logs, syslog, journald, journalctl, rsyslog, syslog-ng, Loki, Promtail, Grafana, alertmanager, alert rules, queries, LogQL, query language, search, filter, regex, parser, label, stream, retention, archive, centralitzat, SIEM, ELK, Elasticsearch, Logstash, Kibana, Graylog, Splunk, Sumo Logic, Datadog, New Relic, Honeycomb, Lightstep, OpenTelemetry, OTLP, span, trace, context, propagator, instrumentation, AIDE, tripwire, file integrity, hash, baseline, compare, scan, update, report, OSSEC, Wazuh, Suricata, Snort, Zeek, Bro, network IDS, host IDS, HIDS, NIDS, malware, virus, rootkit, chkrootkit, rkhunter, ClamAV, clamscan, freshclam, signature, heuristic, anomaly, baseline, deviation, pattern, behavior, signature, IOC, indicator of compromise, TTP, tactics, techniques, procedures, MITRE ATT&CK, kill chain, Lockheed, recon, weaponization, delivery, exploit, install, C2, actions, persist, lateral, evade, cred, discover, collect, exfil, impact, threat hunting, hypothesis, evidence, hunt, analysis, IOC, YARA, Sigma, Elastic rule, detection rule, alert, false positive, true positive, triage, severity, priority, response, escalation, communication, disclosure, GDPR, LOPDGDD, APDCAT, AEPD, notificar, 72h, breach notification, evidence, preservation, chain of custody, forensic, The Sleuth Kit, TSK, Autopsy, Volatility, memory, disk, image, hash, SHA, MD5, integrity, write blocker, bit-stream, dd, ddrescue, dcfldd, Guymager, FTK Imager, EnCase, X-Ways, Magnet AXIOM, Cellebrite, MSAB, mobile, extraction, decoding, analysis, timeline, super timeline, Plaso, log2timeline, reporting, executive summary, technical, findings, recommendations, mitigation, recovery, restore, reimagine, rebuild, certificate, rotation, key rotation, secret, password, passphrase, MFA, 2FA, isolation, quarantine, sandbox, containment, eradication, sanitization, secure delete, shred, dd, scrub, NIST 800-88, purge, clear, destroy, degauss, physical destruction, shredder, incinerator, audit trail, log retention, chain of custody, immutable, write once, WORM, S3 Object Lock, Azure Blob Immutable, compliance, GDPR, LOPDGDD, HIPAA, PCI DSS, SOX, ISO 27001, NIST 800-53, NIST CSF, CIS Controls, OWASP, SANS Top 20, PCI, PA-DSS, QSA, auditor, audit report, remediation, gap analysis, maturity model, CMMI, COBIT, ITIL, ISO 27002, ISO 27005, risk management, risk register, risk assessment, likelihood, impact, exposure, residual risk, risk treatment, accept, mitigate, transfer, avoid, control, detective, preventive, corrective, compensating, deterrent, recovery, control objective, statement of applicability, SoA, ISMS, PDCA, plan, do, check, act, continuous improvement, kaizen, lean, agile, dev sec ops, shift left, security by design, secure by default, defense in depth, Zero Trust, least privilege, separation of duties, dual control, two-person integrity, four-eyes, job rotation, mandatory vacation, background check, security awareness, training, education, phishing simulation, tabletop exercise, red team, blue team, purple team, capture the flag, CTF, bug bounty, responsible disclosure, coordinated disclosure, CVE, CNA, vendor, patch, update, fix, hotfix, security advisory, security bulletin, PGP signed, vendor, third-party, supply chain, attack chain, attack vector, attack surface, threat model, STRIDE, PASTA, VAST, LINDDUN, attack tree, misuse case, abuse case, security requirement, security control, security policy, security standard, security procedure, security guideline, security baseline, security architecture,

security design, security review, security audit, security assessment, security test, penetration test, pen test, vulnerability assessment, threat assessment, risk assessment, compliance audit, internal audit, external audit, third-party audit, attestation, certification, accreditation, authorization, ATO, IATO, IATT, RMF, NIST 800-37, FedRAMP, DoD, IL, impact level, MAC, classified, secret, top secret, TS, SCI, NOFORN, ORCON, REL TO, ITAR, EAR, export control, dual-use, encryption, ECCN, 5D002, Wassenaar, fundamental research, open publication, dual-use research, DURC, ePPP, export, sanctions, OFAC, BIS, DDTC, foreign person, deemed export, technology transfer, TAA, technical assistance agreement, MLA, manufacturing license agreement.

Capítol 50 — DRP: pla de recuperació davant desastres

"Les còpies sense un pla de recuperació són diners al calaix. Cal saber com recuperar-se."

50.1 Què és un DRP

Un **DRP** (Disaster Recovery Plan, pla de recuperació davant desastres) és el document que defineix **com recuperar el sistema** després d'un desastre. Inclou:

- Tipus de desastres previstos.
- Procediments de recuperació pas a pas.
- Responsables.
- Temps objectiu de recuperació (RTO).
- Punt objectiu de recuperació (RPO).
- Recursos necessaris.
- Proves periòdiques.

50.2 RTO i RPO

Dues mètriques clau:

- **RTO** (Recovery Time Objective): quant temps pot estar el sistema caigut? Per exemple, 4 h.
- **RPO** (Recovery Point Objective): quantes dades podem perdre? Per exemple, 24 h (l'última còpia).

Per al BernatLab:

- **RTO**: 4-8 h. Pots tolerar un parell d'hores sense servei.
- **RPO**: 24 h. Còpies diàries amb restic. Si perds 24 h de dades, és acceptable.

Si necessites RTO de 5 minuts, cal **replicació en temps real** (molt car). Per a un homelab, 4-8 h és raonable.

50.3 Tipus de desastres

Tipus que cal preveure:

1. **Disc dur falla**. Més probable del que voldríem. La microSD falla.
2. **Raspberry robada o danyada**. Possible si està accessible.
3. **Tall de llum prolongat**. Més probable a la zona rural.
4. **Incendi, inundació, o desastre natural**. Improbable però devastador.
5. **Atac informàtic**. Possible si hi ha bretxa.
6. **Error humà**. Esborrament accidental, mala configuració.

7. **Actualització que falla.** De vegades passa.

50.4 El pla per al BernatLab

Defineixo un pla pas a pas per a cada desastre:

Escenari 1: la microSD falla

Síntomes: la Raspberry no arrenca, dades corruptes.

Recuperació:

1. **Substituir la microSD** per una de nova (32 GB, classe A2).
2. **Descarregar la imatge** de Raspberry Pi OS Lite des de raspberrypi.com.
3. **Flashear** amb Raspberry Pi Imager o balenaEtcher.
4. **Configurar la Wi-Fi i SSH** abans d'arrencar.
5. **Iniciar sessió** via SSH.
6. **Restaurar la còpia** de restic:

```
sudo apt install restic
restic -r /mnt/usb/backups/bernatlab restore latest \
  --target /home/bernat
```

1. **Reinstal·lar serveis** (Docker, Tailscale, etc.).
2. **Restaurar volums Docker** (InfluxDB, Grafana, etc.).
3. **Verificar** que tot funciona.

Temps estimat: 2-3 hores.

Escenari 2: la Raspberry robada o cremada

Recuperació:

1. **Comprar una Raspberry nova** (o una de segona mà).
2. **Seguir els passos 2-9 de l'escenari 1.**
3. **Recuperar Tailscale** amb la clau d'autenticació.
4. **Recuperar la IP** (pot canviar si tens MagicDNS).

Temps estimat: 1-2 dies (esperant el hardware).

Escenari 3: tall de llum prolongat

Recuperació:

1. **Esperar** que torni la llum.
2. **Si tens UPS**, la Raspberry continua funcionant. Verificar que no s'ha apagat.
3. **Si la Raspberry s'ha apagat**, simplement tornar a engegar.
4. **Verificar** que els serveis s'han reiniciat correctament.

Temps estimat: 5-15 min.

Escenari 4: desastre natural (incendi)

Recuperació:

1. **Acceptar la pèrdua** de l'equip.
2. **Comprar nou hardware**.
3. **Restaurar des de còpies al núvol** (Backblaze, Wasabi, etc.).
4. **Reinstal·lar serveis**.

Temps estimat: 1-2 setmanes.

Escenari 5: atac informàtic

Recuperació:

1. **Aïllar** la Raspberry de la xarxa.
2. **Restaurar des d'una còpia neta** (que sàpigues que no està compromesa).
3. **Canviar totes les contrasenyes**.
4. **Aplicar pegats** de seguretat.
5. **Verificar** que no queda rastre de l'atacant.

Temps estimat: 4-24 h.

50.5 Llista de coses a recuperar

Un **checklist** de recuperació:

- ☐ Còpia de restic del sistema.
- ☐ Còpia de les dades dels volums Docker.
- ☐ Còpia de les configuracions (/etc).
- ☐ Còpia de les claus SSH i Tailscale.
- ☐ Còpia dels secrets (.env).
- ☐ Còpia del repo del BernatLab.
- ☐ Llista de paquets instal·lats (`apt list --installed > packages.txt`).
- ☐ Documentació de la configuració de xarxa.
- ☐ Claus de Tailscale, 2FA, contrasenyes (a Bitwarden).
- ☐ Notes personals sobre configuracions específiques.

50.6 Com provar el DRP

Un pla que no es prova, no funciona. Cal practicar:

Prova de taula (tabletop)

Assegut a la taula, simules un desastre i tries el procediment. Identifiques buits en el pla.

Prova de recuperació

Un cop l'any, restaura el sistema en un hardware diferent (una Raspberry vella, un PC amb Debian, una màquina virtual). Això valida que:

- Les còpies funcionen.
- Els temps de recuperació són correctes.
- La documentació és completa.

Prova de failover

Si tens redundància, prova a fer failover (canviar al sistema de backup). Això valida que el backup funciona realment.

50.7 Eines de recuperació

Tingues sempre a mà:

- Una **microSD** amb Raspberry Pi OS Lite pre-instal·lada.
- Un **disc USB** amb les còpies de restic/Borg.
- Un **ordinador portàtil** amb què puguis accedir a la Raspberry.
- Les **contrasenyes** a Bitwarden (no només a la memòria).
- La **documentació** impresa (un cop l'any) en paper, en un lloc segur.
- Una **còpia impresa** de les claus de Tailscale (recovery codes).

50.8 Redundància

Si vols un sistema que **no es pugui caure**, cal **redundància**:

- **Dues Raspberry** sincronitzades (una activa, una standby).
- **Un NAS** a casa amb còpies automàtiques.
- **Un servidor cloud** (VPS petit) per a serveis crítics.

Això és car i complicat. Per a un homelab, **una bona còpia al núvol + RTO de 4-8 h** és prou.

50.9 Documentació del DRP

Crea un fitxer `DRP.md` al repo amb:

- Tipus de desastres.
- Procediments pas a pas.
- Responsables (Bernat, en aquest cas).

- Contactes d'emergència.
- Recursos externs (cloud, suport, etc.).

Exemple breu:

```
# Pla de Recuperació Davant Desastres (DRP) – BernatLab

## Responsables
- Tècnic principal: Bernat Mora
- Contacte d'emergència: [telèfon]
- Cloud: BernatMora a Backblaze B2

## RT0: 4-8 h
## RPO: 24 h

## Procediments

### Pèrdua de la Raspberry
1. Hardware nou: Raspberry Pi 4, microSD 32GB, font d'alimentació
2. Imatge: Raspberry Pi OS Lite 64-bit
3. Restaurar còpia de restic des de /mnt/usb/backups
4. Reinstal·lar serveis segons el llistat a README
5. Verificar amb Uptime Kuma

### Robatori / pèrdua total
1. Còpies a Backblaze B2
2. Còpies a NAS d'un familiar (configurat)
3. Hardware nou, restaurar igual que "pèrdua de Raspberry"

### Bretxa de seguretat
1. Veure runbook a SEURETAT.md
2. Aïllar el sistema
3. Restaurar des de còpia neta
4. Rotar tots els secrets

## Recursos
- Còpies al núvol: b2:bernatlab-backups
- Recovery codes Tailscale: a Bitwarden
- Master key Bitwarden: a [adreça física]
```

50.10 Eines útils per al DRP

- **Restic / BorgBackup**: còpies (Cap 45).
- **Tailscale**: accés remot segur.
- **Ansible / NixOS / Docker Compose**: permeten reproduir el sistema fàcilment.
- **Bitwarden**: emmagatzematge segur de secrets.
- **Backblaze B2 / Wasabi**: còpia al núvol.
- **Notion / Obsidian**: documentació.
- **GitHub**: repositori de codi i documentació.

50.11 Errors habituals

Error 1: còpies sense prova de restauració.

Si mai has restaurat, no saps si funciona. Prova-ho.

Error 2: còpies al mateix lloc.

Si tens les còpies a la mateixa Raspberry que es pot cremar, no serveixen. Còpia al núvol sempre.

Error 3: documentació obsoleta.

Si el pla no reflecteix l'estat actual del sistema, no serveix. Mantén-lo actualitzat.

Error 4: no practicar.

Si no proves el pla regularment, no funcionarà quan calgui.

Error 5: secrets al DRP.

Si poses contrasenyes al DRP en text pla, són accessibles a tothom. Usa referències ("veure Bitwarden").

50.12 Resum

Un DRP ben fet és la diferència entre recuperar-se ràpid i perdre-ho tot. Identifica els possibles desastres, escriu procediments pas a pas, defineix RTO i RPO, i practica regularment. La combinació de còpies al núvol + runbooks clars + proves periòdiques és la millor garantia de continuïtat. Això tanca el Mòdul 5. En el Mòdul 6 veurem com mantenir tot això en marxa 24/7.

50.13 Exercicis pràctics

1. Inventaria tots els actius del BernatLab.
2. Defineix RTO i RPO raonables.
3. Escriu un DRP amb els escenaris principals.
4. Crea una còpia al núvol (Backblaze B2 o similar).
5. Fes una prova de restauració en un entorn de test.
6. Documenta el procediment de recuperació.
7. Crea una còpia impresa de les claus de recuperació.
8. Programa una revisió anual del DRP.